Contents

1 Introduction

This chapter establishes the foundational context and rationale for the thesis. Section 1.1 outlines the inherent security risks associated with monolithic kernel device drivers and the industry's increasing reliance on complex hardware accelerators. Building upon this problem statement, Section 1.2 evaluates existing literature concerning hardware isolation and program analysis. It demonstrates *why* current mitigation strategies fall short for stateless multimedia pipelines, creating the necessary interconnection that justifies our novel driver extraction methodology.

1.1 Motivation

Device drivers in the modern Linux ecosystem act as the critical bridge between physical hardware and high-level software abstractions. While their modularity allows for dynamic loading without system reboots [CRKH05], this convenience masks a fundamental security flaw: drivers execute with full kernel privileges. Consequently, a single vulnerability within a driver can enable an attacker to compromise the entire host system, bypassing all internal operating system boundaries [Sil26].

This architectural risk is heavily exacerbated by the evolution of mobile devices into hubs for augmented reality, 4K media processing, and on-device artificial intelligence [WLW+17]. Because traditional CPUs have reached their physical limits regarding clock speed and thermal dissipation—a phenomenon known as the failure of Dennard scaling [EBSA+12]—the industry has aggressively shifted toward heterogeneous processing.

Hardware accelerators compute specific tasks with extreme efficiency. However, they require highly complex, often proprietary software stacks and privileged device drivers with Direct Memory Access (DMA) to system memory. This complexity drastically widens the system attack surface. The primary motivation of this thesis is to address this vulnerability: we aim to understand the intricate dependencies of these complex drivers and investigate how to fully extract them from the monolithic kernel to execute safely on bare metal.

1.2 Related Work

This section reviews existing literature on hardware multimedia acceleration, operating system-level driver isolation, and program analysis for reverse engineering. By analyzing the results of these prior works, we highlight their limitations when applied to complex, stateless multimedia pipelines, thereby establishing the relevance of our bare-metal extraction approach.

1.2.1 Hardware Acceleration and the Accelerator Attack Surface

Hardware acceleration is a strict requirement for modern video coding standards such as HEVC/VVC and AV1 due to their immense computational complexity [BWC+21]. However, the deep integration of highly complex hardware drivers into the monolithic Linux kernel poses severe security risks.

While historical vulnerability research heavily focused on central OS components or GPU drivers, modern mobile devices integrate dozens of less-audited, highly privileged accelerators (e. g., VPUs, NPUs, and DSPs) that remain accessible from untrusted user-space contexts. Maar et al. mapped this fragmented attack surface across major OEMs, resulting in the discovery that media and accelerator drivers represent a massive, easily reachable target for attackers, as illustrated in Figure 1.1 [MDS+24].

Category	Device Driver's Entry	Module	Accessibility per OEM Vendors						
			Samsung	Xiaomi	Asus	Realme	OnePlus	Oppo	Vivo
AI	/dev/apuext	apusys.ko	2	12		12		29	40
DSP	/dev/adsprpc-smd	frpc-adsprpc.ko	48	52	83	56	71	43	40
DSP	/dev/fastrpc-[acs]dsp	frpc-adsprpc.ko		10					
NPU	/dev/vertex	npu.ko	38						
AI	/dev/apusys	apusys.ko		4					
Audio	/dev/xlog	xlogchar.ko		60					
GPU Extention	/proc/ged	ged.ko	14	38	17	38	29	57	60
Monitor	/proc/perfmgr	mtk_perf_ioctl.ko	7	38	17	38	29	57	60
JPEG	/proc/mtk_jpeg	jpeg-driver.ko	2	25	17		14	29	40
Memory	/proc/secmem	trusted_mem.ko	12	25					
Monitor	/proc/mi_log	mi_log.ko		21					
Camera	/proc/camera	camera.ko	10						

Figure 1.1: Accessible kernel attack surface from untrusted security contexts, detailing the percentage of Android OEM devices permitting user-space access to hardware drivers (adapted from [MDS+24]).

The severity of this accelerator attack surface is frequently demonstrated in the wild. For example, recent vulnerability research by Google Project Zero resulted in a zero-click exploit chain on the Pixel 9. The attacker bypassed the SELinux media sandbox by exploiting the hardware-accelerated AV1 decoding driver (`/dev/bigwave`) to achieve arbitrary kernel execution [JSF26]. Similarly, critical vulnerabilities in Qualcomm's video accelerator drivers (e. g., CVE-2022-22087) and MediaTek's DSPs

have repeatedly allowed local privilege escalation. These results underscore *why* completely extracting and isolating hardware drivers from the kernel context is a critical security objective for this thesis.

1.2.2 Automated Driver Extraction and Isolation Techniques

To mitigate the risk of single drivers compromising the entire operating system, modern secure systems are shifting toward robust hardware-software isolation.

A prominent historical strategy involves over-approximating the hardware and software environments to run drivers inside emulators. For instance, the *Avatar* framework forwards I/O operations between a physical device and a virtual machine to facilitate dynamic security analysis [ZBFB14]. While this results in an excellent environment for fuzzing, full emulation introduces severe latency, making it *not* suitable for real-time multimedia decoding.

More recent approaches aim to isolate drivers using architectural boundaries. Frameworks like *LXDs* (Lightweight Execution Domains) execute drivers in restricted, separate kernel address spaces [NBJ+19], while *KSplit* utilizes static analysis to automatically generate synchronization glue-code for isolated driver domains [HND+22]. Furthermore, recent research into Neural Processing Units (NPUs) successfully resulted in viable microkernel decomposition and IOMMU-backed sandboxing with minimal performance overhead [J+].

However, applying these concepts to complex, stateless multimedia drivers remains a significant challenge. Unlike simple network cards or isolated NPUs, codec drivers depend heavily on high-bandwidth memory translations and incredibly intricate, high-frequency ioctl handshakes between the user-space Hardware Abstraction Layer (HAL) and the kernel's V4L2 framework. Our work tackles this complexity directly by extracting the V4L2/M2M logic to a standalone bare-metal environment, bypassing the need for in-kernel wrappers entirely.

1.2.3 Program Analysis for Reverse Engineering and Porting

Analyzing and extracting monolithic kernel drivers requires navigating extreme code complexity and internal subsystem dependencies. Understanding these interdependent boundaries necessitates advanced static and dynamic program analysis.

Static analysis tools excel at identifying structural patterns across massive codebases. Systems like *SHARD* utilize fine-grained static analysis, resulting in context-aware kernel hardening by restricting control flows [AAFX21]. Recently, semantic query

engines like *CodeQL* have been successfully applied to the Linux kernel to trace complex, cross-entry data flows [W+25]. However, purely static analysis heavily struggles to resolve dynamic function pointer bindings (such as vtables and operations structures) extensively utilized in subsystems like V4L2.

To resolve these static ambiguities, dynamic analysis is required. Modern kernel tracing utilizes extended Berkeley Packet Filter (eBPF) and high-level frontends like *bpftrace*, resulting in safe, low-overhead dynamic introspection of live kernel data structures without modifying the underlying source code.

By combining these two paradigms, this thesis establishes a novel methodology. Rather than relying on blind fuzzing or full-system emulation, our approach utilizes *CodeQL* to structurally map the V4L2 subsystem's data flow, *and* leverages *bpftrace* to intercept polymorphic boundaries in real-time. This dual approach accurately isolates the exact execution path necessary to run the hardware driver autonomously.

1.3 Contribution

1.4 Organization of this Thesis

2 Background

2.1 AV1 Codec

The history of video codecs begins in 1984, though it was the H.261 codec released in 1986 that established the block-based transform coding paradigm still foundational to modern deployment. For the subsequent two decades, the digital media industry was heavily guided by the Moving Picture Experts Group (MPEG). In 2003, their co-developed H.264 (AVC) codec [Ric04] became the de facto language of internet video distributions due to its high compression efficiency and universal hardware adoption.

The H.265 (HEVC) codec was released as a successor to H.264 in 2013 [ITU13], promising a 50% increase in data compression efficiency. However, its widespread adoption was severely hindered by an incredibly fragmented and aggressive patent licensing landscape. This market friction led to the formation of the Alliance for Open Media (AOMedia) in 2015, a consortium of technology enterprises including Google, Amazon, and Netflix. Their explicit objective was to engineer a next-generation, royalty-free, open-source video standard [All15].

The resulting AV1 bitstream specification was finalized in 2018, delivering approximately a 30% compression improvement over H.265 [CMH+20]. Achieving this level of compression performance relies on highly advanced mathematical and computational models. This complexity represents a barrier for general-purpose CPUs, which lack the throughput to decode high-resolution AV1 streams in real time. Consequently, widespread utilization depends entirely on dedicated hardware accelerators embedded within target platforms.

As detailed in Table 2.1, while H.264 remains the most widely deployed codec, its footprint is contracting. Concurrently, AV1 is experiencing a rapid upward trend as dedicated hardware implementation blocks—such as the Video Processing Unit (VPU) integrated into the Rockchip RK3588 SoC—become standard features in modern computing architectures.

Table 2.1: Comparison of Video Codec Market Share (Bitmovin Reports 8 & 9)

Report Edition	**H.264 (AVC)**	**H.265 (HEVC)**	**AV1**
Report 8 (2024/25) [Bit25a]	79%	49%	13%
Report 9 (2025/26) [Bit25b]	65%	41%	16%

2.2 Linux Device Driver Model

The Linux kernel manages hardware devices through a highly layered abstraction model designed to fully isolate architecture-specific engineering from core system execution operations [CRKH05]. For System-on-Chip (SoC) platforms like the Rockchip RK3588, physical peripheral hardware is typically exposed via an internal platform bus and dynamically mapped at boot time using the Device Tree specification [The24i]. The Device Tree provides a structured description of the hardware topology, enabling the kernel to bind matching driver code directly to specific hardware addresses via an explicit `compatible` string (e.g., `"rockchip,rk3588-vdec"` targeting the Hantro video decoder VPU) [The24m].

The lifecycle of a device driver is governed primarily by specialized framework `probe()` and `remove()` entry points. During initialization, the subsystem's `probe()` routine requests and secures all required hardware assets, such as memory-mapped I/O (MMIO) registers, clock gates, power grids, and hardware interrupt signaling paths (IRQs), before registering the driver with higher-level kernel abstraction frameworks [The24l].

To simplify internal lifecycle state tracking and mitigate memory leaks during driver execution failures, modern drivers rely heavily on the managed resources (*devres*) API using allocation routines such as `devm_kzalloc()` and `devm_ioremap_resource()`. These resource managers register memory cleanups within the device framework, automatically unmapping and freeing assets when the driver is unbound [The24d]. While this strategy ensures stability inside Linux, it creates a tight architectural coupling between the driver and the core kernel environment, representing a significant technical hurdle when extracting this driver logic for standalone, bare-metal execution.

2.2.1 Kernel Infrastructure Dependencies

Beyond explicit peripheral allocations, hardware driver routines operate inside an expansive ecosystem of internal kernel services assumed to be globally available at runtime [CRKH05]. These operational dependencies form a complex software boundary

between driver execution routines and the kernel core. Safely executing a driver inside an isolated bare-metal environment requires identifying and emulating these interfaces. Key infrastructure dependencies include:

- **Memory Management:** The core Direct Memory Access (DMA) engine APIs (e.g., `dma_alloc_coherent()`, `dma_map_single()`) abstract system Input-Output Memory Management Unit (IOMMU) translations, handle hardware cache coherency protocols, and arrange physical scatter-gather page layouts [The24e].
- **Synchronization Primitives:** Drivers manage multi-threaded concurrency using internal kernel synchronization lockers, such as spinlocks (`spinlock_t`), mutual exclusions (`struct mutex`), blockable wait queues (`wait_queue_head_t`), and runtime signaling checkpoints (`struct completion`) [CRKH05].
- **Interrupt Handling:** Execution lines like `request_irq()` and `devm_request_irq()` register low-level interrupt service routines with the central kernel IRQ routing architecture, which coordinates physical vector distribution and deferred work routines [CRKH05].
- **Clock and Power Management:** SoC peripherals depend on highly complex clock trees and runtime power gating domains governed by the Common Clock Framework (CCF) and the runtime power management infrastructure. Drivers interact with these configurations using interface requests like `clk_prepare_enable()` and `pm_runtime_get_sync()` [The24b, The24n].

2.3 The V4L2 Media Subsystem

The Video for Linux Two (V4L2) media subsystem is the primary kernel core framework responsible for handling video ingest, transmission, rendering, and hardware codec logic within modern Linux platforms [The24p]. It maps complex multi-stage hardware streaming blocks into standard character device interfaces, fully isolating user-space multimedia engines from lower-level device implementations.

2.3.1 V4L2 Framework Overview

V4L2 functions as the main entry point for constructing multimedia pipelines, exposing underlying device interfaces through accessible paths within the standard virtual file system (e.g., `/dev/videoX`) [CRKH05]. User-space processing code coordinates operations with these device handles via standard POSIX system calls including `open()`, `close()`, `ioctl()`, and `mmap()`.

The core interface layout relies on `ioctl()` parameter commands to negotiate capabilities and structure data configurations. Upon accessing a target device handle, an application evaluates hardware features using the `VIDIOC_QUERYCAP` request, which establishes if the associated peripheral supports capture paths, stream output channels, or complex memory-to-memory operations. Formatting parameters are established using `VIDIOC_G_FMT` and `VIDIOC_S_FMT`, allowing applications to declare target pixel layouts (such as an AV1 compressed stream payload or raw NV12 destination frames), geometric dimensions, and color space coordinate ranges [The24p].

2.3.2 Videobuf2 (VB2) Framework

The Videobuf2 (*vb2*) sub-framework is a highly performance-optimized core asset within V4L2 engineered exclusively to handle high-throughput video buffer allocation, state-tracking, and streaming I/O routines [The24q]. It forms the deterministic state machine that regulates frame buffer ownership as data crosses back and forth between user-space runtimes and the underlying driver hardware.

The operational sequence of an active buffer within the vb2 engine moves through several explicit steps:

1. **IDLE/DEQUEUED:** The buffer memory block is owned entirely by the user-space runtime; it can be populated with fresh bitstream data or read to consume uncompressed video frames.
2. **PREPARED:** The buffer is submitted to the kernel framework via `VIDIOC_PREPARE_BUF`, prompting vb2 to validate boundaries, secure structures, and set up memory table mappings.
3. **QUEUED:** The buffer is inserted into the active incoming processing queue via `VIDIOC_QBUF`, awaiting execution by the hardware driver.
4. **ACTIVE:** The low-level driver extracts the buffer from the vb2 layer queue to execute the actual hardware acceleration job (e.g., feeding bitstream slices to the decoding engine).
5. **DONE:** Hardware processing completes successfully; the buffer is pushed back into the completed vb2 tracking queue, ready to be collected by user-space clients via `VIDIOC_DQBUF`.

To interface across various device layouts and memory architectures, the vb2 engine offers three distinct memory-streaming allocation methods [The24q]:

- **MMAP (Memory Mapping):** Frame buffers are allocated in continuous kernel spaces, and user-space contexts project these buffers into their local virtual addresses using standard `mmap()` system requests.

- **USERPTR (User Pointers):** User-space engines maintain tracking loops and allocate tracking blocks dynamically, passing raw virtual memory vectors down to the kernel. This method forces the kernel to dynamically pin target address pages, which introduces processing overhead.
- **DMABUF (DMA Buffers):** Buffers are allocated directly by an external hardware allocator component (e.g., a DRM/KMS graphics engine memory pipeline) and shared globally via file descriptors. This method enables zero-copy buffer transmission between fully distinct subsystems, which is required for high-throughput VPU pipelines on SoCs like the Rockchip RK3588.

2.3.3 Memory-to-Memory (M2M) Framework

Hardware multimedia accelerators, such as the Hantro VPU on the RK3588 SoC, function simultaneously as data destinations and data sources; they absorb a compressed input bitstream and generate uncompressed output video frames. The V4L2 Memory-to-Memory (*m2m*) framework offers the formal abstraction layer for such operations by managing two distinct vb2 queues concurrently: an *OUTPUT* queue (acting as the input path to the driver/hardware) and a *CAPTURE* queue (acting as the output path from the driver/hardware) [The24o].

The m2m framework isolates concurrent processes via individual execution contexts (`struct v4l2_m2m_ctx`). This multi-instance capability is paired with an internal job scheduler. When user space queues buffers to both the OUTPUT and CAPTURE queues of an instance, the m2m framework marks that context as ready and schedules it. When the context is executed, the framework invokes the driver-specific `device_run()` callback, signaling the underlying driver to program the hardware registers and begin processing [The24o].

2.3.4 Subsystem Layering and Interconnection

The execution topology of a Linux hardware decoding driver is highly hierarchical, routing requests through multiple kernel layers.

User-space actions (such as buffer submission) target the top-level **V4L2 Subsystem** ioctl layer. V4L2 validates the system call parameters and forwards the request down to the **M2M Framework**, which correlates the operation with the correct `v4l2_m2m_ctx` context. The M2M framework then interacts with the **Videobuf2 Framework** to track the buffer's state transitions, manage memory mapping flags, and ensure DMA safety. Once both input and output buffers are secured in their respective queues, the M2M scheduler pushes the job into the low-level **Hardware Driver** layer. The driver translates these abstract kernel structures into raw physical address offsets, configures peripheral registers, and kicks off the execution loop on the physical VPU hardware.

Understanding this strict sequence of data transformations and boundary crossings between V4L2, VB2, M2M, and the lower-level hardware driver is fundamental to establishing the specific, minimal structural environment required to replicate this behavior on bare metal.

2.4 Stateless Video Decoding

Modern System-on-Chip (SoC) video processing units (VPUs) increasingly utilize a stateless hardware architecture. Unlike older generations of video accelerators, stateless accelerators shift the burden of bitstream parsing and state management entirely from the hardware firmware to user-space software and kernel drivers [**?**].

2.4.1 Stateful vs. Stateless Codec Model

In a traditional *stateful* codec model, the VPU contains an embedded microcontroller running proprietary firmware. The user-space application simply feeds a raw, compressed bitstream (such as an H.264 or AV1 stream) into the driver. The internal firmware parses the sequence headers, manages the Decoded Picture Buffer (DPB) for reference frames, tracks the hardware state, and outputs decoded frames [The24k].

Conversely, a *stateless* hardware decoder lacks an onboard microcontroller or firmware capable of parsing high-level stream syntax. The hardware is a pure execution engine. Consequently, the user-space application must parse the bitstream's structural components (e.g., Sequence Header OBUs and Frame Header OBUs in AV1) and extract all high-level syntax elements. This per-frame metadata must be explicitly formatted into kernel-defined configuration structures and passed alongside the compressed slice or tile data directly to the hardware driver [Duf20].

2.4.2 The Request API

Because traditional V4L2 interfaces were designed for synchronous, stream-oriented stateful devices, they inherently lacked a mechanism to bind specific metadata configurations to a particular video buffer atomically. To bridge this gap for stateless hardware, the Linux kernel introduces the *Request API* [The24j].

The Request API allows user space to bundle configuration parameters (V4L2 controls) and data buffers together into an atomic execution unit called a *request*, tracked by a media device file descriptor. The lifecycle of a stateless decode operation typically adheres to the following sequence:

1. **Allocation:** User space allocates a new request object via the `MEDIA_IOC_REQUEST_ALLOC` ioctl on the media controller node.
2. **Configuration:** Frame-specific decode parameters are assigned to the request using the `VIDIOC_S_EXT_CTRLS` ioctl, referencing the request's file descriptor.
3. **Enqueueing:** The target `OUTPUT` (compressed bitstream) and `CAPTURE` (destination frame) buffers are queued to the video device node via `VIDIOC_QBUF`, explicitly bound to the request.
4. **Submission:** The completed request is submitted to the driver via the `MEDIA_REQUEST_IOC_QUEUE` ioctl.

This architecture generates highly complex, high-frequency ioctl sequences for every decoded frame. For bare-metal driver extraction, this poses a substantial challenge: since the Request API is deeply integrated into the V4L2 core and media controller frameworks, a standalone environment must bypass this entire ioctl sequence and directly reconstruct the underlying associations between memory buffers and codec metadata.

2.4.3 AV1 Decode Context

The AV1 video coding standard exemplifies the peak complexity of the stateless decoding model. Compared to older codecs like H.264, AV1 relies on an extensive assortment of dynamic frame-level parameters that must be evaluated before hardware scheduling [CMH+20].

To support stateless AV1 decoding, the Linux V4L2 framework exposes specialized, highly nested control structures that the user-space parser must populate for every frame [Lin26]. These include:

- `struct v4l2_ctrl_av1_sequence`: Contains global sequence parameters including profile, bit depth, and chroma subsampling configurations.
- `struct v4l2_ctrl_av1_frame_header`: Encapsulates extensive frame-level parameters, including quantization matrices, loop filter coefficients, segmentation maps, global motion vectors, and tile grids.
- `struct v4l2_ctrl_av1_film_grain`: Contains the parameters necessary for synthesis of film grain, a feature executed as a post-processing step in the AV1 pipeline.

Furthermore, AV1 uses advanced dynamic context modeling based on Cumulative Distribution Functions (CDFs) for entropy coding, alongside complex reference frame indices where a single frame can reference up to eight prior anchors [CMH+20]. The driver must program these physical pointer addresses into the VPU registers

accurately. The sheer volume and tightly coupled nature of these AV1-specific structures demonstrate why a bare-metal extraction requires an exact, comprehensive mapping of driver structures: omitting or misconfiguring even a single flag within these multi-kilobyte metadata payloads will lead to register corruption or immediate hardware hangs.

2.5 Kernel Polymorphism and Analysis Challenges

Although written in procedural C, the Linux kernel extensively relies on object-oriented programming paradigms to achieve polymorphism, allowing core subsystems to interact with hardware-specific drivers without hardcoding vendor logic [CRKH05]. This architecture is primarily implemented using structures of function pointers, which function identically to a C++ Virtual Method Table (vtable).

2.5.1 Operations Structure Pattern

The Videobuf2 framework implements this pattern to decouple generic buffer state transitions from low-level hardware register configurations [The24q]. The core framework defines an operations structure interface, which the low-level driver populates with pointers to its own concrete implementations. As shown in Listing 2.2, `struct vb2_ops` establishes the contract, which the Hantro driver fulfills by binding its internal functions during compilation.

```
struct vb2_ops {
    int (*buf_prepare)(struct vb2_buffer *vb);
    void (*buf_queue)(struct vb2_buffer *vb);
    // ...
};

static const struct vb2_ops hantro_qops = {
    .buf_prepare = hantro_buf_prepare,
    .buf_queue = hantro_buf_queue,
    // ...
};
```

Listing 2.2: Polymorphism via function pointer structures in Videobuf2.

2.5.2 The Resolution Challenge

While this abstraction benefits kernel modularity, it presents a fundamental challenge for static code analysis, source-code navigation, and automated reverse-engineering. When the Videobuf2 core triggers a buffer operation, the invocation is executed indirectly through the structure reference:

```
ops->buf_prepare(vb);
```

Because this reference is decoupled at compile time, traditional static analysis tools—such as text-based `grep` or Language Server Protocol (LSP) indexing definitions—cannot resolve this abstract invocation to its concrete implementation (`hantro_buf_prepare()`) [**?**]. An LSP "go-to-definition" command on `ops->buf_prepare()` merely redirects the analyst to the structural declaration in the global kernel headers (`videobuf2-core.h`), completely obscuring the vendor-specific code path.

The actual binding between the framework and the driver happens dynamically at runtime during device initialization, when the driver registers `&hantro_qops` into the active `vb2_queue` struct. Consequently, mapping the exact execution paths across the V4L2, VB2, and Hantro boundaries requires more than literal code parsing. It necessitates a combination of precise semantic analysis to manually trace structure instantiation data flows, and dynamic analysis (e.g., runtime function tracing) to capture actual symbol resolutions as they execute on the hardware platform.

2.6 Static Analysis with CodeQL

CodeQL is a semantic code analysis engine that treats source code as a relational database, allowing researchers to query complex control- and data-flow patterns using a declarative logic programming language. For C/C++ projects like the Linux kernel, CodeQL parses the source code into an Abstract Syntax Tree (AST) and exposes it through standard library classes [Git26b].

2.6.1 CodeQL Fundamentals and AST Interactions

A CodeQL query is built upon logical functions called *predicates*, which encapsulate reusable conditions and return boolean values [Git26a]. For example, predicates can filter elements by file path strings or specific function names.

To trace specific execution patterns within the kernel, CodeQL queries traverse various AST nodes:

- **`Function` and `Call`:** Represent function declarations and direct function invocations, respectively.
- **`ExprCall` and `PointerFieldAccess`:** Used to model indirect invocations, such as calling a function through a pointer nested inside a structure field (e.g., `ops->queue_setup(...)`).

- `FunctionAccess:` Represents scenarios where a function pointer is passed as an argument to another function.
- `ClassAggregateLiteral:` Models the static initialization of a C structure. This is particularly critical in kernel analysis for resolving polymorphic operations structures (as discussed in Section 2.5.1), because it links the abstract `Field` of a `Struct` to its concrete implementation [Git26b].

Queries heavily rely on the `exists()` formula for existential quantification, which introduces local variables to establish relationships between different AST nodes (e.g., mapping a specific `PointerFieldAccess` to its corresponding `ClassAggregateLiteral`). Additionally, CodeQL provides performance directives such as `bindingset`, which restricts the evaluation of a predicate until specific arguments are bound to a concrete value, preventing unbounded and computationally expensive database traversals.

2.6.2 Call Graph Resolution and Recursion

Constructing an accurate call graph in the Linux kernel requires mapping both direct calls and resolved indirect calls. Once a base predicate is defined to establish a single parent-child call relationship, CodeQL can apply a *transitive closure* operation (denoted by the `+` or `*` operators) [Git26a]. The `*` operator computes the recursive closure, allowing the analysis to find all deeply nested functions reachable from a specific entry point (e.g., tracing from a V4L2 ioctl down to the hardware driver).

To manage the massive amount of data generated by recursive call graph queries, CodeQL supports aggregations. Query results can be grouped, counted, or concatenated into sets, which significantly improves the visualization and readability of the output when mapping complex kernel subsystem boundaries.

2.6.3 Variable Access and Data Flow Analysis

Beyond control flow, extracting a standalone driver requires identifying which internal kernel structures are mutated during execution. CodeQL provides `VariableAccess` nodes to track where a specific variable or structure field is read or written [Git26b].

However, simple structural access tracking has limitations: it tracks direct syntactic references but struggles if the variable is passed through multiple intermediate assignments, pointer arithmetic, or wrapper functions. To overcome this, CodeQL offers Global Data Flow and Taint Tracking libraries [Git26a]. Taint tracking models how data propagates from a specific *source* (e.g., a user-space ioctl argument) to a *sink* (e.g., a hardware register write). By analyzing the flow of tainted data

rather than just literal variable accesses, the analysis can accurately identify all kernel objects and fields that influence the driver's execution state, even across deep functional boundaries.

2.7 Dynamic Kernel Instrumentation

While static analysis provides a structural map of the kernel and its drivers, dynamic instrumentation is required to observe actual runtime behavior, validate code execution paths, and capture live register configurations without modifying or recompiling the running kernel [Gre19b].

2.7.1 eBPF Overview

Extended Berkeley Packet Filter (eBPF) is an in-kernel virtual machine that allows developers to execute sandboxed programs within the Linux kernel space dynamically [The24f]. Originally designed for network packet filtering, modern eBPF serves as a general-purpose infrastructure for system tracing, observability, and security.

The core architecture of eBPF prioritizes absolute kernel stability and safety. Before an eBPF program can be loaded into the kernel, it must pass through the *eBPF Verifier* [Tha25]. The verifier performs strict static analysis on the bytecode to guarantee that the program cannot cause a kernel crash; it ensures the code contains no infinite loops, respects strict memory boundaries, and avoids out-of-bounds pointer dereferences. Once validated, a Just-In-Time (JIT) compiler translates the generic eBPF bytecode into native architecture-specific machine instructions (e.g., ARM64 for the RK3588) [The24f]. This execution flow provides near-zero runtime overhead, making eBPF safe and highly performant for tracing latency-sensitive video processing pipelines.

2.7.2 Kprobes and Function Tracing

Kernel Probes (*kprobes*) represent a foundational mechanism for dynamic kernel tracing under Linux [The24h]. A kprobe can be attached to virtually any arbitrary instruction address within the core kernel or loaded modules. When a probe is registered, the kernel copies the target instruction, replaces it with a software breakpoint or an architecture-specific trapping instruction, and executes a user-defined callback whenever that code path is hit [The24h].

Despite their power, kprobes suffer from significant structural limitations imposed by compiler optimizations. Modern compilers aggressively employ *function inlining* when building monolithic kernels. When a function is inlined, its distinct entry

point and stack frame are eliminated, and its body is directly woven into the calling function. Because the symbol table no longer retains an address entry for inlined blocks, standard kprobes cannot attach to them directly. For complex driver stacks like V4L2, this requires researchers to manually identify non-inlined outer wrapper functions or shift focus to alternative tracing targets to catch elusive execution steps.

2.7.3 bpftrace

`bpftrace` is a high-level, domain-specific tracing language built on top of the eBPF and BCC ecosystems [Gre19b]. Inspired by AWK and DTrace, it provides a highly concise syntax to declare dynamic probes, aggregate statistics, and introspect deep kernel data structures on the fly.

A critical element that makes modern `bpftrace` invaluable for kernel analysis is BPF Type Format (BTF) [The24a]. BTF is a compact metadata format that encodes C type information, struct layouts, and field offsets directly inside the kernel binary. Historically, tracing deep nested structure pointers required downloading gigabytes of unstripped kernel debugging symbols (`CONFIG_DEBUG_INFO`). BTF provides a lightweight alternative, enabling `bpftrace` to understand and safely access structure fields directly at runtime without external symbol files [The24a].

This capability is vital for overcoming the kernel polymorphism challenges described previously. While static tools struggle to resolve where a function pointer like `ops->buf_queue` actually points, a short `bpftrace` script can intercept the call boundary in real time and print the exact symbol name of the target function, resolving structural ambiguity instantly as shown in Listing 2.3. This precise method was utilized to validate the active runtime execution of core driver interfaces like `hantro_queue_setup` and the primary M2M scheduling interface `device_run`.

```
/* Tracing the low-level vb2 queue operations */
kprobe:vb2_core_qbuf
{
    $q = (struct vb2_queue *)arg0;
    /* Print the address and resolved symbol of the dynamic queue setup op */
    printf("Active vb2_ops target: %s\n", ksym($q->ops->queue_setup));
}
```

Listing 2.3: Example bpftrace script resolving dynamic function pointer targets.

3 Implementation

This chapter presents a three-stage methodology for analyzing the kernel-driver boundary interactions in the Hantro VPU driver for AV1 decoding on the Rockchip RK3588 SoC. The approach combines complementary analysis techniques, each addressing limitations of the previous:

1. **Manual analysis** (Section 3.2) establishes a verified ground truth call graph for a single ioctl (`VIDIOC_REQBUFS`), providing complete visibility into direct and indirect function calls within the media subsystem.
2. **Static analysis** (Section 3.3) scales this approach using CodeQL to discover all possible execution paths across the driver, including indirect calls through operations structures that standard tools cannot resolve.
3. **Dynamic analysis** (Section 3.5) validates which statically identified paths actually execute during real AV1 decode workloads, distinguishing active code from unreachable over-approximations.

Manual tracing is exhaustive but does not scale beyond single code paths; static analysis provides complete coverage but includes theoretical paths that may never execute [MS18][NNH99]; dynamic analysis confirms execution but may miss edge cases not triggered by test workloads. By triangulating across all three methods, this methodology achieves both completeness (from static analysis) and accuracy (from dynamic verification)[Ern03], with the integrated results presented in Chapter 4.

Before any analysis could proceed, a functional hardware and software environment was required. Section 3.1 describes the establishment of this research platform, including the resolution of non-trivial boot challenges and the integration of a modern mainline-aligned kernel with hardware-accelerated AV1 decoding capabilities.

3.1 Establishing the Research Environment

The Radxa Rock 5B, featuring the Rockchip RK3588 SoC, served as the primary hardware target. This platform was selected for its native silicon-based AV1 decoding capabilities, despite the significant integration challenges posed by custom kernel development.

3.1.1 Hardware Initialization and Power Constraints

The Radxa Rock 5B exhibits stringent and non-trivial power requirements, particularly during the transition from U-Boot to kernel initialization[Rad26b]. Empirical testing revealed that the RK3588 SoC's transient power spikes during the boot sequence frequently exceed the current limits of standard Power Delivery (PD) adapters and integrated host ports. As summarized in Table 3.1, stable operation was highly dependent on the power source's ability to negotiate specific voltage rails and sustain peak current draws. The Aukey QC 3.0 adapter provided stable operation for all experiments in this thesis.

Table 3.1: Power Supply Compatibility Observations

Power Source	**Type / Rating**	**Result**
Aukey QC 3.0 (USB-A)	18W (up to 12V/1.5A)	**Stable**
Digitec GaN (USB-C)	65W (Multi-profile)	**Stable**
ThinkPad TB4 Port	Thunderbolt 4 (Host)	**Stable**
Apple USB-C Adapter	20W PD	**No Power**
ThinkPad USB-A 3.2	Integrated PC Port	**Boot Loop**
ThinkPad Slim Charger	65W (Fixed PD)	**Failed**

3.1.2 Kernel Integration: OpenCCA and AV1 Support

To enable realistic analysis of modern driver architectures, the default vendor OS (utilizing a downstream 6.1 kernel) was replaced with the OpenCCA kernel (version 6.12)[BS25]. This kernel provides greater adherence to the mainline Linux architecture, which is essential for accurate driver behavior analysis.

The build process leveraged the vendor-provided Board Support Package (BSP) toolchain[Rad26a]. To maintain compatibility with the existing build infrastructure, a symbolic link was created to map the OpenCCA configuration (`defconfig`) to the expected `rockchip_defconfig`, ensuring the build scripts correctly identified the target parameters.

3.1.3 Kernel Configuration and Instrumentation

To support deep kernel introspection and dynamic tracing, the kernel build configuration was augmented to expose the system's internal execution state. The default BSP build system employs a rigid mechanism that overlays configuration fragments from `/bsp/linux/rk2410/`, often overwriting manual changes. To ensure custom requirements persisted, these parameters were injected into the `./0001-common/` configuration scripts, guaranteeing their priority during the build process.

The following instrumentation and debugging support was enabled to facilitate the dynamic analysis described in Section 3.5:

- **Symbol Resolution (`KALLSYMS_ALL`):** Maps all kernel symbols—including internal variables—to system memory [Lin24]. This is critical for resolving raw addresses to human-readable function names during stack traces and for enabling comprehensive kprobe coverage.
- **Dynamic Tracing (`Ftrace`, `Kprobes`):** Enables dynamic instrumentation to record function call graphs [Ros26] and insert runtime probes [MPK+06]. This provides a non-invasive method for monitoring the execution path of the Hantro driver without requiring constant recompilation.
- **Introspection (`BPF`, `BTF`):** The integration of BPF and BPF Type Format (BTF) enables modern, type-safe debugging. BTF allows debuggers and tracing tools to interpret complex C structures (structs/unions) directly from the kernel binary [The24a], eliminating the need for manual offset calculation via `pahole` [CdM23].

These additions provide the necessary visibility to perform the runtime verification methodology described in Section 3.5. A comprehensive list of the enabled kernel configuration flags is provided in Appendix A.1.1.

3.1.4 Bootloader Configuration and Serial Console Debugging

Compiled images were deployed to the Rock 5B via `scp` over a local network. To manage the co-existence of the custom OpenCCA kernel and the original vendor kernel, a dual-boot configuration was maintained within the `extlinux.conf` configuration file.

During initial deployment, a "silent boot" phenomenon was observed: while the system appeared to hang after U-Boot execution, network analysis confirmed that the kernel had initialized successfully and was accessible via SSH. The lack of serial output was traced to a driver mismatch in the `append` line of the boot configuration. The default vendor parameters utilized `console=ttyFIQ0,1500000n8`, which relies on the Rockchip-specific Fast Interrupt Request (FIQ) debugger [Roc24]. This driver is supported in the vendor's downstream 6.1 kernel but requires specific alignment of the FIQ logic and kernel hooks that were not prioritized in the OpenCCA 6.12 build.

3.1.5 GStreamer Build and Configuration

To enable hardware-accelerated AV1 decoding, a custom build of GStreamer 1.24.11 was required. The system-provided Debian 12 repositories (version 1.22.9) lack the

necessary stateless AV1 support for the RK3588. GStreamer 1.24.11 was selected specifically for its support of the `v4l2slav1dec` element [GSt24], which utilizes the V4L2 Request API [The24j]. This element exposes the kernel-driver boundary for tracing via `strace` [Str24] and `eBPF`, providing the observability necessary for both the manual analysis phase (Section 3.2) and dynamic verification (Section 3.5).

The build process using meson required strict version consistency across all source packages (`core`, `plugins-base`, `good`, and `bad`). To minimize unnecessary dependencies on the headless target, `-Dauto_features=disabled` was used as a baseline, with only essential elements enabled. The complete build configuration and dependency resolution is detailed in Appendix A.1.3.

During integration, the kernel's Contiguous Memory Allocator (CMA) [The24c] posed a potential bottleneck for the VPU DMA engine, which requires large, contiguous memory for frame buffers. Although allocation warnings occurred during early stress testing, the final 6.12 kernel configuration proved stable for 1080p decoding using default memory settings. However, manual tuning (e.g., `cma=256M`) remains necessary for 4K streams or highly fragmented systems.

Installation was verified by confirming the presence of the `V4L2 Stateless AV1 Video Decoder` element via `gst-inspect-1.0` and executing a test pipeline (Listing A.1.4) against a 1080p AV1 stream. Successful termination with `Got EOS` confirmed end-to-end hardware decoding functionality.

3.2 Manual Analysis using LSP

To establish a verified ground truth for the driver's execution flow, a comprehensive manual trace of the `VIDIOC_REQBUFS` ioctl [The24g] was performed. This ioctl was selected as the logical entry point for analysis, as it initiates the memory allocation and buffer management lifecycle that underpins all subsequent decode operations. The manual trace provides a reference baseline against which the static and dynamic analysis results can be validated.

The scope of this manual trace was deliberately constrained to functions within the media subsystem—specifically, the V4L2 core, Videobuf2 framework [The24q], M2M helper layer [The24o], and the Hantro driver itself. Calls outside this subsystem (e.g., into the memory management or DMA subsystems) were noted but not recursively traced. A complete list of included source files is provided in Chapter 4, Section 4.1.

3.2.1 Development Environment: NeoVim and LSP Integration

To facilitate efficient navigation of the extensive kernel source code, NeoVim [Neo24] with the `clangd` language server (Version 18.1) [LLV24] was configured as the primary development environment, leveraging the Language Server Protocol (LSP). Because the kernel is compiled for the ARM64 architecture, `clangd` required explicit configuration to recognize cross-compilation headers and flags. A compilation database (`compile_commands.json`) was generated using the kernel's internal utility script, `scripts/clang-tools/gen_compile_commands.py`, providing the language server with the exact compiler flags and include paths used during the build process. The complete Clang configuration, including flag pruning to remove GCC-proprietary options that are incompatible, is documented in Appendix A.1.2.

3.2.2 Manual Tracing of the Callgraph

To map the execution path of the AV1 decoding process, a recursive manual tracing methodology was employed. The entry point into the kernel was identified dynamically using `strace` [Str24], a userspace utility that intercepts system calls. By filtering the output of the GStreamer decoding command (Listing A.1.4) for I/O Control (`ioctl`) operations, the `VIDIOC_REQBUFS` ioctl was isolated as the initial trace target.

While the LSP-enabled environment facilitated the navigation of direct function calls and structure definitions, the Linux kernel's heavy reliance on object-oriented patterns presented a significant challenge. Specifically, the kernel utilizes "operations" structures (e.g., `struct vb2_mem_ops`, `struct vb2_ops`) to achieve polymorphism through function pointers. This abstraction prevents standard LSPs from resolving indirect calls, such as `vb2_queue->ops->buf_prepare`, since the specific implementation is determined at runtime during driver initialization.

To resolve these indirect branches, two complementary approaches were employed. First, the driver source files were manually audited to identify where these operation structures were instantiated—for example, locating the initialization of `hantro_qops` (of type `struct vb2_ops`) to determine which function implements the `buf_prepare` callback. Second, as the manual trace progressed and additional operation structures were encountered, a simple CodeQL query was developed to systematically search for all implementations of a given function pointer type (Section 3.3.3). These two approaches—manual inspection and automated discovery—overlapped during the trace, with manual findings informing the CodeQL query design and CodeQL results accelerating subsequent manual navigation.

The resulting callgraph for `VIDIOC_REQBUFS` encompasses approximately 100 functions, documenting the flow from the initial ioctl request through the Videobuf2

(vb2) framework and into the driver-specific hardware logic. This trace is visualized via Mermaid-based flow diagrams [ST26] (Section **??**). While this manual approach established a complete and verified ground truth for a single ioctl, extending this methodology to the remaining 14 ioctls in the target set (listed in Chapter 4) would require prohibitive effort. Section 3.3 describes how CodeQL static analysis automated this discovery process across the full driver scope.

3.3 Static Analysis with CodeQL

To scale the manual tracing approach across the full set of target ioctls, static analysis was performed using CodeQL [Git24], a semantic code analysis engine. CodeQL facilitates the discovery of complex patterns and callgraph relationships by treating source code as a relational database. This approach was specifically selected to address the challenge of tracing indirect function calls within the Linux kernel; unlike traditional search tools, CodeQL allows for semantic data-flow analysis to resolve implementation patterns across the extensive kernel codebase.

The static analysis methodology over-approximates the execution paths by including all theoretically reachable functions, regardless of whether they execute in practice. This over-approximation is intentional: it ensures completeness by avoiding false negatives (missing functions that do execute) at the cost of including false positives (functions that never execute). Section 3.5 describes the dynamic verification approach used to distinguish active execution paths from this static superset.

3.3.1 Database Generation and Scoping

The efficacy of CodeQL depends on a relational database generated during the compilation process. The CodeQL CLI intercepts the build commands to observe the exact compiler flags, include paths, and macro definitions used for each source file. Because the extraction process is resource-intensive, the database was generated on an `x86_64` host machine using the ARM64 cross-compilation toolchain. To ensure a performant analysis environment, the build was scoped specifically to the subsystems relevant to AV1 decoding:

```
codeql database create hantro-db \
    --language=cpp \
    --overwrite \
    --source-root=bsp/.src/linux \
    --command="make ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu- -j$(nproc) \
    drivers/media/v4l2-core/ \
    drivers/media/common/videobuf2/ \
    drivers/media/platform/verisilicon/"
```

Listing 3.2: CodeQL database creation with scoped kernel build

As shown in Listing 3.2, the build was restricted to the V4L2 core, the Videobuf2 framework, and the `verisilicon` directory (the Hantro driver location). This scoping accurately reflects the architecture-specific macros and conditional compilation blocks relevant to the Rockchip RK3588 SoC, while excluding unrelated drivers and subsystems that would inflate the database size and introduce spurious results.

3.3.2 Developing the Hantro QL Library

To modularize the analysis and simplify complex queries, a custom QL library (`hantro_lib.qll`) was developed. This library encapsulates common logic into reusable predicates accessible via the `Hantro::` namespace. Basic predicates include:

- `isDriverFile(File f)`: Validates if a file is located within the Hantro (`verisilicon`) directory.
- `isSubsystemFile(File f)`: Checks if a file belongs to the core media subsystems (V4L2, Videobuf2, or the driver).
- `isV4l2Ioctl(Function f)`: Identifies if a function matches the specific ioctls identified during the dynamic tracing phase (Section 3.2.2).

3.3.3 Resolving the Call Graph: Direct and Indirect Invocations

The primary objective of the static analysis is to construct a recursive call graph originating from an initial `ioctl` entry point. Handling the Linux kernel's execution flow requires addressing two types of invocations.

Direct Calls Standard function calls are resolved using CodeQL's `Call` class. A relationship is established where the enclosing scope of an invocation matches the caller function, mapping it to its statically known target.

Indirect Calls The kernel uses structure-based function pointers (e.g., `struct vb2_ops`) to achieve polymorphism. Resolving a call like `vb2_queue->ops->buf_prepare` is impossible through standard static resolution, as the specific implementation is determined by which operations structure instance is assigned at runtime. However, CodeQL represents global designated initializers as `ClassAggregateLiteral` objects. A custom predicate was developed to map function pointers back to these initializations (Listing 3.3.3).

Additionally, the kernel frequently passes function pointers as arguments to higher-order functions—for example, `hantro_return_bufs(q, v4l2_m2m_dst_buf_remove)`, where the second argument is a callback that will be invoked by the callee. To capture this pattern, the `calls` predicate includes a third case that identifies when a function is passed by reference as an argument (Case 3 in Listing 3.3).

```
predicate calls(Function caller, Function callee) {
    // Case 1: Direct function calls
    exists(FunctionCall site |
        site.getEnclosingFunction() = caller and
        site.getTarget() = callee
    )
    or
    // Case 2: Indirect calls via ops struct function pointers
    exists(ExprCall indirect, Field f |
        indirect.getEnclosingFunction() = caller and
        resolvesFunctionPointerCall(indirect, f, callee)
    )
    or
    // Case 3: Function pointer passed as argument
    // e.g., hantro_return_bufs(q, v4l2_m2m_dst_buf_remove)
    exists(FunctionCall site, FunctionAccess fa |
        site.getEnclosingFunction() = caller and
        fa = site.getArgument(_) and
        callee = fa.getTarget()
    )
}
```

Listing 3.3: CodeQL predicate cases for resolving function calls

The discovery of Case 3 emerged during the validation phase (Chapter 4, Section 4.3), when cross-comparison with the manual trace revealed functions that were definitively executed but not captured by the initial static analysis. This iterative refinement—using discrepancies between methods to improve query completeness—demonstrates the value of the triangulated approach.

The Over-Approximation Challenge Combining direct and indirect logic yields a unified `calls(caller, callee)` predicate. However, because the database includes the entire V4L2 core, querying an initializer for a common structure like `v4l2_ioctl_ops` returns implementations for every driver in the scope. This creates an over-approximated tree containing hardware paths irrelevant to the hardware AV1 decoding; for example, implementations for other codecs never execute.

3.3.4 Filtering Indirect Calls through Iterative Completeness

To isolate the active Hantro execution path, a filter predicate `isImplementedFunction` was introduced. This predicate restricts the call graph to functions positively identified as part of the Hantro AV1 decoding pipeline. To ensure no implementation paths were missed during this filtering, an iterative "Discovery" query was implemented. This query identifies all indirect calls that branch to "untracked" structures:

1. Identify entry functions via `isV4l2Ioctl`.
2. Traverse the call graph using `callsUnfiltered*` (unfiltered transitive closure).
3. Isolate indirect calls via `resolvesFunctionPointerCall`.
4. Filter out results where the struct is already handled by `isTrackedOpsStruct`.

If this query returns results, it indicates a gap in the filter logic—specifically, a new operations structure used by the driver that has not yet been accounted for. The results are manually audited to determine which implementations belong to the Hantro driver, and the relevant structure type is added to the `isTrackedOpsStruct` predicate. The filter is considered "complete" when this query returns zero results across the entire analysis scope, indicating that all operations structures in the active call graph have been identified and tracked. The full implementation of this discovery query is provided in Appendix A.2.

While this iterative approach ensures completeness within the static analysis, it still produces an over-approximation: functions that are statically reachable but never execute during actual workloads remain in the call graph. Section 3.5 describes the dynamic verification methodology used to confirm which functions are actually invoked during AV1 decoding.

3.4 Cross-Boundary Struct Access Analysis

Identifying the kernel data structures that the Hantro driver touches at runtime is a prerequisite for defining the emulation surface. A naïve enumeration of all struct references in the driver source over-approximates this set by including dead-code paths and compile-time constants never active during a decode session. Two complementary static analysis strategies were therefore implemented in CodeQL, each capturing a different dimension of the boundary: one establishes a complete syntactic upper bound, and one restricts that bound to verified data-flow paths.

3.4.1 Method 1: Syntactic Field Access Analysis

The syntactic approach identifies every location in the ioctl-reachable call graph where a function in one domain directly accesses a field of a struct defined in the other. Because it operates purely over AST relations without data-flow overhead, it is fast and complete: any struct touched anywhere on any reachable path is included. Domain classification reuses the file-path predicates defined in `hantro_lib` (Section 3.3.2), and reachability is bounded by the same transitive call graph used throughout the analysis (Section 3.3.3).

```
from Function entry, Function reached, Struct s, string direction
where
  Hantro::isV4l2Ioctl(entry) and Hantro::calls*(entry, reached) and
  not s.getName() = "" and
  exists(string funcDomain, string structDomain |
    funcDomain  = getDomain(reached.getFile()) and
    structDomain = getDomain(s.getFile())    and
    funcDomain != structDomain               and
    direction   = funcDomain + " -> " + structDomain)
select s.getName()
```

Listing 3.4: Syntactic cross-boundary field access query (simplified)

The limitation of this approach is precision. It flags any reachable function that textually references a cross-domain struct, regardless of whether that reference lies on an active control path. Codec layout constants accessed only within a single hardware variant's decode path, for example, appear alongside the core buffer management structs, producing a result that must be treated as an upper bound rather than a minimal emulation surface.

3.4.2 Method 2: Global Taint Tracking

The taint tracking approach replaces syntactic co-occurrence with verified pointer lineages: a struct is included only when a pointer to it can be shown to transit the domain boundary along a traceable execution path from a defined source to a defined sink. This requires enumerating how the boundary is actually crossed at runtime.

3.4.2.1 Three Crossing Patterns

Examination of the V4L2/VB2 subsystem architecture (Section **??**) reveals three structurally distinct ways in which a kernel struct pointer enters driver code.

Kernel → Driver (K→D): The kernel invokes a driver callback registered through `vb2_ops`, `v4l2_ioctl_ops`, or `v4l2_m2m_ops` and passes a kernel struct pointer as a

direct argument. The source is therefore the parameter of the driver callback function, typed as a kernel struct pointer; the sink is any field access on that parameter within the driver.

Driver Query → Kernel (DQ→K): The driver actively retrieves a kernel struct pointer by calling a kernel accessor such as `hantro_get_ctrl`. This pattern is architecturally necessary in the stateless codec model (Section **??**): because per-frame decode parameters arrive through the V4L2 Request API rather than as callback arguments, the driver must pull them explicitly before each hardware invocation. Without a dedicated configuration for this pattern, all codec-specific control structs obtained through this path would be invisible to the K→D configuration. The source here is the return value of a kernel function called from driver code; the sink is any field access in driver code on the returned pointer.

Driver → Kernel (D→K): The driver passes a pointer to a driver-owned struct into a kernel function. The source is that argument at the call site; the sink is any field access in kernel code on the received pointer.

Separating these three patterns into independent `TaintTracking::Global` configurations is necessary because their source and sink definitions are mutually exclusive: combining them into a single configuration would require sources that span both directions simultaneously, which produces spurious flows.

3.4.2.2 Additional Flow Steps

Standard intra-procedural taint propagation does not cover all relevant patterns. Four custom flow steps are registered uniformly across all three configurations via a shared `sharedFlowStep` predicate.

Qualifier propagation ensures that taint flows from a pointer to any field access through that pointer (`ptr` $\rightsquigarrow$ `ptr->field`), which is not automatic when the sink is bound to the qualifier expression rather than the `FieldAccess` node itself.

Function pointer dispatch delegates to `resolvesFunctionPointerCall` in `hantro_lib`, propagating taint across statically resolved ops callbacks (Section 3.3.3).

Structural `container_of` traversal handles the common Linux pattern of recovering an outer struct pointer from an embedded member pointer. The naive approach — invoking `getAnExpandedElement()` over all `container_of` macro expansions — creates a Cartesian product join across every AST node in every expansion and does not terminate on a kernel-scale codebase. Instead, the query navigates the fixed AST layout that `container_of` always expands to: a GNU statement expression whose first declaration is the `__mptr` temporary variable. The cast-stripped initialiser of that variable is the original pointer argument, giving a direct one-to-one mapping.

```
exists(MacroInvocation mi, StmtExpr se, BlockStmt blk,
      DeclStmt ds, Variable mptr, Expr ptrArg |
  mi.getMacroName() = "container_of" and mi.getExpr() = se and
  se.getStmt() = blk and blk.getStmt(0) = ds and
  ds.getADeclaration() = mptr and mptr.getName().matches("__mptr%") and
  ptrArg = mptr.getInitializer().getExpr().(Cast).getExpr() and
  n1.asExpr() = ptrArg and n2.asExpr() = se)
```

Listing 3.5: Structural container_of flow step

Union field traversal handles the `v4l2_ctrl_ptr` union through which `v4l2_ctrl` exposes codec-specific payload. Named pointer fields of this union (`p_av1_frame`, `p_hevc_sps`, and similar) are matched by a prefix predicate, allowing taint to flow into concrete codec struct types without enumerating them individually.

3.4.2.3 Bounding and Scope Control

Without scope restriction, taint tracking over a kernel-scale database is prohibitively expensive and admits spurious paths through unrelated subsystems. The `isReachableFromIoctl` predicate (Section 3.3.2) restricts both sources and sinks to functions in the call graph slice that executes during a V4L2 ioctl session. An important secondary effect is that probe-time functions (`hantro_probe`, `hantro_attach_func`) are excluded: their kernel struct accesses — clock handles, device-tree match tables, media framework topology objects — represent one-time hardware initialisation rather than runtime data dependencies, and should not appear in the emulation surface derived from the decode path. The DQ→K configuration applies this bounding to its source predicate specifically, since its sources originate entirely within driver code and would otherwise admit any driver function that calls a kernel accessor, regardless of whether it is reachable from an ioctl.

Table 3.6: Comparison of analysis methods

Property	**Syntactic Field Access**	**Global Taint Tracking**
Analysis basis	AST node co-occurrence	Pointer lineage across domains
Runtime	∼2 s	∼20 s
Precision	Low (includes dead paths)	High (active data transit only)
Completeness	High (conservative bound)	Partial (flow-break sensitive)
Probe-time structs	Included	Excluded by bounding

Table 3.6 summarises the trade-offs. The two methods are deliberately complementary: the syntactic query provides the complete candidate set, and the taint analysis assigns verified crossing patterns to the subset it can trace. Structs present in the syntactic result but absent from the taint result indicate either dead-code paths (true negatives) or flow-graph gaps such as unresolved function pointer chains (false negatives to be addressed by dynamic verification, Section **??**).

3.5 Dynamic Verification using bpftrace

While the static analysis provides complete coverage of all theoretically reachable execution paths, it necessarily includes functions that may never execute during actual AV1 decoding workloads. To validate which paths are active under real operating conditions, dynamic analysis was performed using **bpftrace** [Gre19a]. Bpftrace is a high-level tracing language for Linux eBPF (extended Berkeley Packet Filter) that abstracts the complexity of **kprobes**, allowing developers to dynamically instrument kernel function entries and execute custom logic at runtime.

This dynamic verification serves two critical purposes: First, it confirms the assumptions of the structs that are implemented by the AV1 decoding; second, it identifies functions that are actually invoked during decode operations, eliminating false positives from the over-approximated call graph. The integration of static and dynamic results is described in Section 3.6.

3.5.1 The Role of BTF vs. Manual Offsets

Modern Linux kernels support **BTF (BPF Type Format)** [The24a], which provides the eBPF subsystem with embedded knowledge of kernel structures. This allows bpftrace to dereference fields like `$struct_ptr->ops->function` directly by name, eliminating the need for manual offset calculation.

In environments without BTF support, developers must manually calculate memory offsets for every struct member using tools like `pahole` [CdM23]. This manual offset calculation is tedious and error-prone, as even a minor kernel version change can shift struct alignments and invalidate the trace script. For this project, ensuring BTF support (via `CONFIG_DEBUG_INFO_BTF=y`, as described in Section 3.1) was a prerequisite for reliable dynamic verification. Combined with `CONFIG_KALLSYMS_ALL=y`, which ensures that all kernel symbols—including static functions—are available for tracing, the instrumented kernel provided comprehensive visibility into the driver's execution.

3.5.2 Handling Inlined Functions and Pointer Dereferencing

A basic bpftrace one-liner, such as `bpftrace -e 'kprobe:function_name { @calls = count(); }'`, is sufficient for tracing non-inlined functions. However, functions marked with the `__inline__` attribute or inlined by the compiler lack distinct entry points in the kernel's symbol table, making them invisible to standard kprobes.

An initial attempt was made to create an automated script by extending the CodeQL query to extract argument types and positions for every function, with the goal of tracing the *caller* of the inlined function and dereferencing the pointers from there.

This approach ultimately proved unreliable, as the callers themselves were frequently inlined by the compiler, creating a recursive inlining problem that obscured the call stack.

Consequently, a simpler manual approach was adopted for inlined functions. By intercepting a non-inlined function that accepts a relevant structure as an argument, the pointer chain could be traversed to the `ops` field, and the function pointer dereferenced to identify the implementation. With BTF and `CONFIG_KALLSYMS_ALL` enabled, this manual dereferencing approach became significantly more effective, as structure layouts and symbol addresses were directly accessible (Listing 3.7).

```
#include <linux/media/videobuf2-core.h>

kprobe:target_kernel_function
{
    /* With BTF and CONFIG_KALLSYMS_ALL, we can resolve implementation pointers */
    $struct_ptr = (struct vb2_queue *)arg0;
    $func_ptr = $struct_ptr->ops->buf_queue;

    printf("Implementation resolved to: %s\n", ksym($func_ptr));
}
```

Listing 3.7: bpftrace example for resolving function pointers via BTF

3.5.3 Automated Trace Generation and Manual Supplementation

To scale the verification across the large set of functions identified by static analysis, a Python script was developed to automate the generation of bpftrace probes. The workflow proceeded as follows:

1. **CodeQL Export:** The static call graph was exported from CodeQL [Git24], listing all functions identified as part of the execution path.

2. **Traceability Check:** The Python script cross-referenced each function name against `/sys/kernel/debug/tracing/available_filter_functions` on the Rock 5B (requiring `sudo` access). This file lists all functions that are available for dynamic tracing via kprobes.

3. **Probe Generation:** For functions present in the available filter list, the script generated a bpftrace probe following the template shown in Listing 3.8.

4. **Manual Flagging:** Functions absent from the available filter list—typically due to inlining or being marked `static` without `KALLSYMS_ALL`—were flagged and appended as comments in the generated script.

5. **Manual Supplementation:** The flagged functions were then handled manually using the pointer-dereferencing approach described in Section 3.3.4, tracing a non-inlined caller and resolving the function pointer at runtime.
6. **Execution:** All probes—both automated and manual—were executed during active AV1 decode workloads (using the GStreamer pipeline from Listing A.1.4).
7. **Result Collection:** Hit counts for each probe were collected, confirming which functions executed during the decode operation. These results are analyzed in Chapter 4.

```
kprobe:hantro_buf_queue
{
    @vb2_ops["hantro_buf_queue calls"] = count();
}
```

Listing 3.8: Example of a generated kprobe for implementation counting

This combined pipeline—automated generation for traceable functions and manual scripts for untraceable cases—allowed for comprehensive runtime verification of the static call graph. The distinction between functions that are statically reachable and functions that actually execute is critical for the extraction methodology described in subsequent chapters, as it identifies the minimal set of kernel objects that must be emulated for bare-metal driver operation.

3.6 Integration and Validation Strategy

The three analysis approaches described in this chapter address different aspects of completeness and accuracy:

- **Manual analysis** (Section 3.2) provides a verified ground truth with complete visibility into the execution flow, but limited coverage (one ioctl of 15 in the target set).
- **Static analysis** (Section 3.3) provides complete coverage of all theoretically reachable code paths, but includes over-approximations (functions that are statically reachable but never execute).
- **Dynamic analysis** (Section 3.5) provides execution confirmation under real workload conditions, but may miss edge cases not triggered by the test workloads (e.g., error paths or rarely used codec features).

To establish confidence in the findings, the methodology triangulates across all three approaches:

1. **Core Set:** Functions identified by all three methods—manually traced, statically reachable, and dynamically confirmed—form the high-confidence core of the execution path. These functions are definitively required for driver operation.
2. **Static Over-Approximations:** Functions found by static analysis but not confirmed by dynamic tracing are flagged for manual verification. These may represent error handling paths, codec-specific logic not exercised by the AV1 test workload, or genuinely unreachable code.
3. **Dynamic Discoveries:** Functions that execute during dynamic tracing but were not predicted by static analysis indicate gaps in the CodeQL query logic—for example, indirect calls through mechanisms not captured by the `ClassAggregateLiteral` resolution approach. Such discoveries prompt refinement of the static analysis.

This triangulation approach ensures both completeness (from static analysis) and accuracy (from dynamic verification). The integrated results, including the final call graph, execution statistics, and identification of critical kernel objects requiring emulation, are presented in Chapter 4.

4 Results

This chapter presents the integrated results of the three-stage analysis methodology described in Chapter 3. Section 4.1 defines the analysis scope and lists the complete set of target ioctls. Section 4.3 validates the methodology by comparing the manually traced ground truth against static analysis results, identifying and categorizing discrepancies. Section **??** presents the complete, validated call graph for all target ioctls. Section **??** analyzes dynamic execution patterns, distinguishing active code paths from unreachable over-approximations. Finally, Section **??** identifies the critical kernel objects and abstractions that must be emulated for bare-metal driver extraction.

4.1 Analysis Scope and Target Ioctls

The analysis focused on the media subsystem components directly involved in AV1 hardware decode operations. The scope was deliberately constrained to the following source directories:

- **V4L2 Core:** `drivers/media/v4l2-core/` (ioctl dispatch, control framework, device registration)
- **Videobuf2 Framework:** `drivers/media/common/videobuf2/` (buffer lifecycle management, memory backends)[The24q]
- **Hantro Driver:** `drivers/media/platform/verisilicon/` (hardware-specific decode logic, DMA programming)

Calls outside this subsystem—such as DMA API invocations (`dma_alloc_coherent`), memory management (`alloc_pages`), or interrupt handling (`request_irq`)—were noted as boundary crossings but not recursively traced.

The complete set of 15 target ioctls comprises:

1. `VIDIOC_QUERYCAP` - Device capability query
2. `VIDIOC_ENUM_FMT` - Format enumeration
3. `VIDIOC_G_FMT` - Get format
4. `VIDIOC_S_FMT` - Set format

5. `VIDIOC_REQBUFS` - Request buffers (ground truth trace)
6. `VIDIOC_QUERYBUF` - Query buffer status
7. `VIDIOC_QBUF` - Queue buffer for decode
8. `VIDIOC_DQBUF` - Dequeue completed buffer
9. `VIDIOC_STREAMON` - Start streaming
10. `VIDIOC_STREAMOFF` - Stop streaming
11. `VIDIOC_QUERYCTRL` - Query control
12. `VIDIOC_S_EXT_CTRLS` - Set extended controls (frame metadata)
13. `MEDIA_REQUEST_IOC_QUEUE` - Submit request
14. `MEDIA_REQUEST_IOC_REINIT` - Reinitialize request
15. `DMA_BUF_IOCTL_SYNC` - DMA-BUF synchronization

4.2 Call graphs

The following figure 4.1 helps illustrate the different subsystems and how they interact. It was created using the calls between the files of the callgraph of the ioctl `REQBUFS` The next figure 4.2 illustrates the full call graph of the `REQBUFS` ioctl. The clusters in figure 4.2 depict the different subsystems based on the file where the function was defined. The most obvious thing to observe is the interconnection between the subsystems as they mostly call each other. In the simplified figure 4.1 these connections become clear: the core interaction is between the hantro driver and the vb2 core. The rest of the files function as helpers who dont do any backcalls. This aligns well with the fact that the `REQBUFS` ioctl uses the vb2 core to allocate buffers and uses the driver mostly to probe capabilities and specifications on buffer size and similar.

4.3 Methodology Validation: Manual vs. Static Analysis

To validate the static analysis methodology, the manually traced ground truth for `VIDIOC_REQBUFS` (174 functions, described in Section 3.2.2) was systematically compared against the CodeQL-generated call graph for the same ioctl. This comparison revealed 7 major discrepancies, which were categorized by root cause and resolved through iterative refinement of both the manual trace and the static analysis queries.

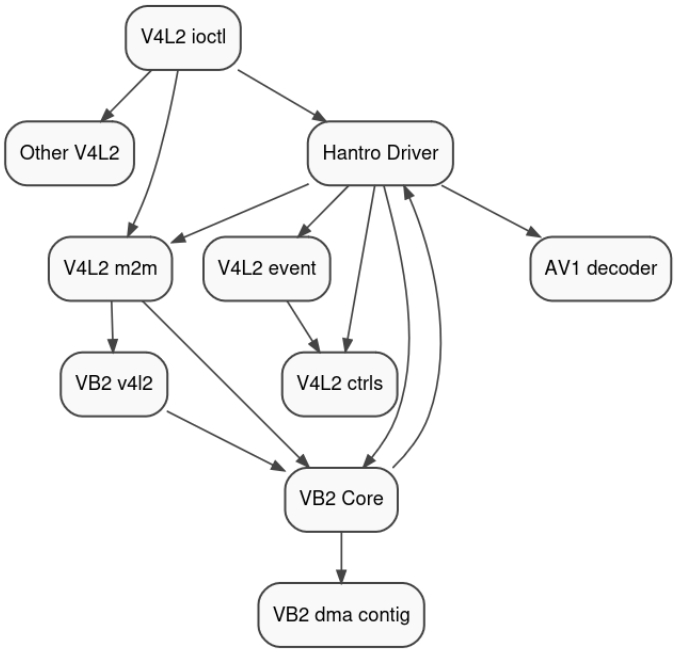

Figure 4.1: A simplified representation of the calls between files of the three Subsystems

4.3.1 Discrepancy Taxonomy

Comparing the manual architectural trace with the automated CodeQL static analysis revealed several layers of divergence. The discrepancies fell into distinct categories, ranging from human error and query gaps to fundamental differences in how humans and compilers interpret source code abstractions.

4.3.1.1 Category 1: Conditional Check Misinterpretation (Manual Error)

Functions: `fh_to_ctx`, `vidioc_g_fmt_out_mplane`

Root Cause: During manual tracing, conditional checks of the form `if (ops->function)` were misinterpreted as actual function invocations. These checks test for the *existence* of a function pointer (to determine whether optional functionality is available) but do not execute the callback. Static analysis correctly identified these as data accesses rather than call graph edges.

Resolution: Removed from the manual trace.

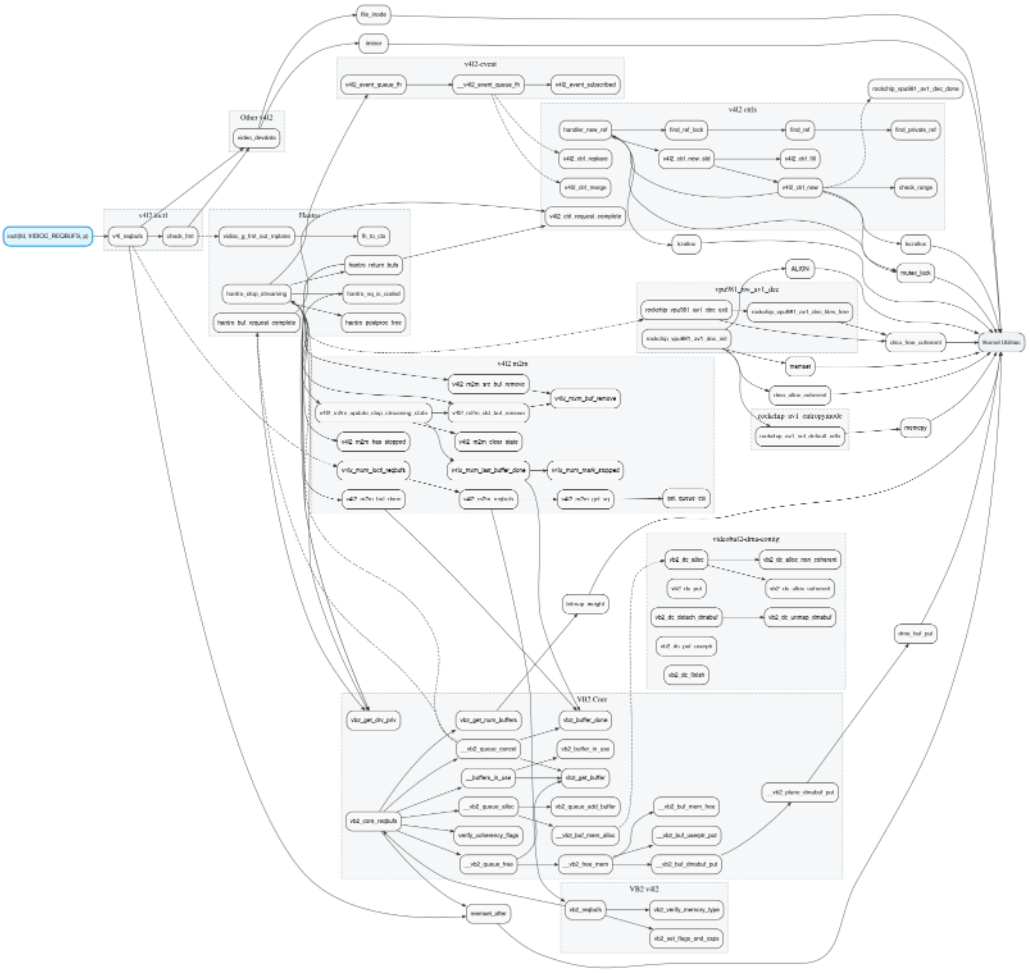

Figure 4.2: Visualisation of the calltree of the VIDIOC_REQBUFS ioctl

4.3.1.2 Category 2: Scope Mismatch (Initialization vs. Runtime)

Functions: `rockchip_vpu981_av1_dec_init`, `rockchip_av1_set_default_cdfs`, `vb2_core_dqbuf`

Root Cause: These functions belong to different operational phases of the driver lifecycle:

- `rockchip_vpu981_av1_dec_init` and `rockchip_av1_set_default_cdfs`: Driver initialization/probing phase. These are called once during module load and are entirely disjoint from the `VIDIOC_REQBUFS` runtime execution.
- `vb2_core_dqbuf`: Belongs to the buffer dequeue path (`VIDIOC_DQBUF`), which is a separate ioctl with no logical entry point during buffer allocation.

Resolution: Removed from the `VIDIOC_REQBUFS` trace. *Lesson:* Clearly delineating analysis scope (initialization vs. runtime, per-ioctl boundaries) is critical for preventing cross-contamination of call graphs.

4.3.1.3 Category 3: Static Analysis False Negative (Query Gap)

Function: `v4l2_m2m_src_buf_remove`

Root Cause: The function was passed as a function pointer argument to another routine (e.g., `hantro_return_bufs(q, v4l2_m2m_src_buf_remove)`), where it is stored and invoked later. The initial CodeQL query only captured direct calls and ops struct invocations, missing this higher-order function pattern.

Resolution: Added a specific case to the CodeQL `calls` predicate (Listing 3.3) to identify function pointers passed as arguments. Re-running the query successfully captured this function. *Lesson:* Iterative query refinement based on manual ground-truth validation is essential for automated completeness.

4.3.1.4 Category 4: Typographical Errors and Categorization

Functions: `v4l2_ioctl_ops`, `v4l_reqbuf`

Root Cause:

- `v4l2_ioctl_ops`: Categorization error. This is a configuration structure (a table of function pointers), not an executable function. Static analysis correctly excludes non-function declarations.
- `v4l_reqbuf`: Typographical error in manual trace naming. The actual implemented macro/function is `v4l_reqbufs` (with a trailing 's').

Resolution: Removed `v4l2_ioctl_ops` from the functional trace and corrected the spelling of `v4l_reqbufs`.

4.3.1.5 Category 5: Preprocessor Macros and Inline "Pseudo-Functions"

Functions: `container_of`, `ARRAY_SIZE`, `BIT_ULL`, `sizeof`, `WARN_ON`, `is_valid_ioctl`

Root Cause: The manual trace captured human-readable developer semantics. However, these are preprocessor macros or inline evaluations that do not exist as concrete callable entities in the compiled binary. Consequently, CodeQL's query engine bypasses them, as they do not generate valid `FunctionCall` edges.

Resolution: Filtered from the final architectural comparison. *Lesson:* Semantic code review tracks *intent*, while static analysis tracks *compilation reality*.

4.3.1.6 Category 6: Subsystem Aliasing and Implementation Obfuscation

Functions: `dev_err` vs. `_dev_err`, `kzalloc` vs. `kzalloc_noprof`

Root Cause: The Linux kernel heavily aliases base implementations for debugging, logging, and profiling.

- **Logging:** The manual trace logged the source-level variadic macros (e.g., `dev_err`, `pr_warn`), while the static analysis captured the underlying compiler targets (`_dev_err`, `_printk`).
- **Memory Allocation:** The manual trace recorded standard API calls (`kzalloc`), but static analysis traced down to the un-profiled allocation primitives (`kzalloc_noprof`, `__kvmalloc_node_noprof`).

Resolution: Unified the aliases during the filtering phase to ensure matching 1:1 equivalency.

4.3.1.7 Category 7: Broad vs. Constrained Hardware Scope

Functions: `hantro_h264_dec_exit`, `vb2_vmalloc_alloc`, `vb2_dma_sg_alloc`

Root Cause: The manual trace was contextually constrained to the Rockchip AV1 hardware utilizing DMA Contiguous (`vb2_dc_*`) memory. Conversely, the automated static analysis recursively traversed all function arrays defined in the driver, capturing cleanup routines for entirely different codecs (H.264, VP8) and capturing alternative `videobuf2` memory backends (vmalloc, dma-sg).

Resolution: Subsystem branches unrelated to the target hardware configuration were pruned from the static analysis output to match the manual context.

4.3.2 Validation Summary

Table 4.3 summarizes the core discrepancies and the error sources.

Key Finding: Evaluating the core logical path errors, the vast majority were human errors in the manual trace, while only one represented a structural gap in the static analysis engine. However, aligning the *raw* data sets (174 manual functions vs. 212 static functions) demonstrated that static analysis captures substantial compilation-level noise (macros, hardware aliases) that humans naturally abstract away.

After resolving logical discrepancies, extending the CodeQL queries, and systematically filtering out preprocessor expansions and unexercised hardware branches, the manual and static traces for `VIDIOC_REQBUFS` achieved complete agreement. Both

Table 4.3: Primary Validation Discrepancies Between Manual and Static Analysis

Function	Category	Error Source	Resolution
`fh_to_ctx`	Conditional Check	Manual	Removed from manual
`vidioc_g_fmt_out_mplane`	Conditional Check	Manual	Removed from manual
`rockchip_vpu981_av1...`	Scope Mismatch (Init)	Manual	Removed from manual
`rockchip_av1_set_def...`	Scope Mismatch (Init)	Manual	Removed from manual
`vb2_core_dqbuf`	Scope Mismatch (Runtime)	Manual	Removed from manual
`v4l2_m2m_src_buf_remove`	Query Gap (Pointer Arg)	Static	Added Case 3 to query
`v4l2_ioctl_ops`	Categorization (Struct)	Manual	Removed from manual
`v4l_reqbuf`	Typographical Error	Manual	Corrected to `v4l_reqbufs`
`ARRAY_SIZE`, `WARN_ON`...	Macro/Inline Expansion	Architectural	Filtered from comparison
`kzalloc_noprof`, `_printk`...	Subsystem Aliasing	Architectural	Unified aliases
`hantro_h264_dec_exit`...	Hardware Scope Breadth	Architectural	Pruned dead branches

the refined manual trace and the refined static analysis produced an identical, validated set of 93 core architectural functions.

4.4 Analysis of Callgraph

For this section, the results of the function call trace with all 15 ioctls as entry were simplified to trace between kernel files. This helps to create an overview of the used subsystems without cluttering.

The cross-module call analysis (Table 4.4) reveals the following structurally important findings for a bare-metal extraction of the Hantro AV1 decoder.

V4L2 mem2mem as the highest-fanout module. `v4l2-mem2mem` is the single module with the highest ioctl fanout in the dataset: a single intra-module call edge is reachable from 8 distinct ioctls, and the full intra-module call graph spans all 9 observed ioctls (see Table A.6). This is structurally expected, since `v4l2_m2m_ioctl_*` wrappers forward every standard buffer-management ioctl directly into the mem2mem scheduler. For extraction purposes, *every* one of these entry points must be provided or stubbed.

Media controller as a mandatory dependency. The Media Controller (`mc-request.c`) appears as a caller that reaches into *both* VB2 core and the V4L2 controls subsystem with a maximum reach of 6 ioctls. This is the stateless Request API fence-synchronisation path: `media_request_object_bind` and `v4l2_ctrl_request_setup`

Table 4.4: Aggregated subsystem dependency summary derived from the CodeQL recursive-call analysis across all 15 target ioctls. Each row represents one *callee module*; the *Reached from* column lists every distinct caller module that has at least one call edge terminating inside the callee. **Max reach** is the highest number of distinct ioctls observed on any single aggregated call edge into that callee; **Ioctl union** is the total number of unique ioctls observed across all such edges. Modules that only call themselves (no external callers) are omitted.

Module (callee)	**Reached from (external callers)**	**Max reach**	**Ioctl union**
V4L2 core (dev/common)	Hantro driver (generic), V4L2 controls, V4L2 ioctl dispatch, V4L2 mem2mem, VB2 core	7	11
V4L2 mem2mem	Hantro driver (generic), V4L2 ioctl dispatch	8	9
V4L2 controls	Hantro driver (generic), Media controller, V4L2 ioctl dispatch	6	7
V4L2 event	Hantro driver (generic), V4L2 controls	5	5
Media controller	V4L2 ioctl dispatch	1	1
VB2 core	Hantro (RK AV1 decode), Hantro driver (generic), Media controller, V4L2 mem2mem, VB2 DMA-contig allocator	6	9
VB2 DMA-contig allocator	Hantro (RK AV1 decode), Hantro driver (generic), VB2 core	4	7
MM frame_vector helper	VB2 DMA-contig allocator	2	2
Hantro driver (generic)	Hantro (RK AV1 decode), V4L2 controls, V4L2 ioctl dispatch, V4L2 mem2mem, VB2 core	5	10
Hantro (RK AV1 decode)	Hantro driver (generic)	4	4

are invoked on every `VIDIOC_QBUF` and `VIDIOC_STREAMON`, making the media controller a *mandatory* dependency even though the Hantro decoder has no topology-configurable pipeline.

Bidirectional coupling in the V4L2 controls subsystem. The controls subsystem (`v4l2-ctrls-*.c/h`) is internally dense—16 raw file-pairs, max reach 6—and also *calls back* into `hantro_drv.c` via the `v4l2_ctrl_ops` callback `s_ctrl`, confirming the bidirectional coupling between the control framework and the driver's codec-configuration logic. Both directions must therefore be present in any standalone build.

DMA-contiguous allocator scope. The DMA-contiguous allocator (`videobuf2-dma-contig`) is reached from VB2 core (max reach 4) and indirectly from the Hantro generic layer, but *only* for the buffer-setup ioctls (`REQBUFS`, `QBUF`). The absence of stream-control ioctls (`STREAMOFF`) in its callee set is consistent with the VB2 contract: allocator callbacks are invoked only during buffer allocation and DMA mapping, not during streaming-state transitions.

The `frame_vector` helper as a marginal dependency. `frame_vector.c` (the `mm/` scatter-gather helper) is reached via only 2 ioctls (`QBUF`, `REQBUFS`), exclusively through the chain `videobuf2-dma-contig` → `videobuf2-memops` → `frame_vector`. It is therefore a marginal but non-zero external dependency that must be stubbed or provided in a bare-metal build.

V4L2 event as a non-optional module. `v4l2-event.c` is reached both from the controls layer (max reach 5) and directly from the Hantro generic driver (max reach 2), but only for the streaming ioctls. Because `hantro_drv.c` registers a `v4l2_subscribed_event` for source-change events, this module cannot be omitted from an extraction even if no userspace consumer subscribes to events at runtime.

4.5 Virtual Interface Structures and Hardware-Specific Implementations

The Linux media subsystem heavily relies on object-oriented paradigms implemented in C via structures containing function pointers (commonly denoted as `_ops` structs). These virtual tables decouple the generic kernel framework from hardware-specific driver implementations. During the static analysis trace, 13 core operational structures were identified as relevant to the overall video decoding pipeline.

To accurately restrict the call graph to the Rockchip RK3588 AV1 decoding context, the static analysis query was designed to map these generic `_ops` structures to their concrete implementations within the Hantro driver. The mapped implementations are as follows:

- **`vb2_ops`:** Implemented by `hantro_queue_ops` defined in `hantro_v4l2.c`. It manages the core lifecycle of video buffers.
- **`hantro_codec_ops`:** The hardware-specific codec interface, implemented specifically for the RK3588 AV1 decoder in `rockchip_vpu981_hw_av1_dec.c`.
- **`vb2_mem_ops`:** Implemented by the DMA contiguous memory allocator (`vb2_dc_*`) in `videobuf2-dma-contig.c`. Other memory backends like vmalloc or dma-sg were deliberately omitted.
- **`vb2_buf_ops`:** Implemented by `v4l2_buf_ops` in `videobuf2-v4l2.c`, which manages V4L2-specific buffer validations and initializations.
- **`v4l2_ioctl_ops`:** Implemented by `hantro_ioctl_ops` in `hantro_v4l2.c`, which routes user-space ioctl commands to the driver.
- **`v4l2_ctrl_ops`:** Implemented by `hantro_av1_ctrl_ops` in `hantro_drv.c`, handling AV1-specific V4L2 controls.
- **`v4l2_ctrl_type_ops`:** Implemented in the core `v4l2-ctrls-core.c` framework for type-specific control operations.
- **`v4l2_m2m_ops`:** Implemented by `vpu_m2m_ops` in `hantro_drv.c`, handling the memory-to-memory job scheduling.
- **`hantro_postproc_ops`:** Handles optional post-processing features on the Rockchip hardware.
- **`media_request_object_ops`:** Utilized by two distinct operational structs for unbinding requests (`vb2_req_unbind` and `v4l2_ctrl_request_unbind`).

Three additional structures—**`v4l2_subscribed_event_ops`**, **`v4l2_subdev_video_ops`**, and **`v4l2_subdev_pad_ops`**—were tracked but found to be unreachable or untriggered during standard, simple AV1 decoding operations.

4.5.1 Handling Optional Operations (`NULL` Pointers)

Many functions defined within these `_ops` structures are optional. Driver maintainers frequently leave these function pointers set to `NULL` (e.g., `g_volatile_ctrl` in `v4l2_ctrl_ops`, or `job_abort` and `job_ready` in `v4l2_m2m_ops`). Because they are entirely omitted from the concrete implementation struct, they do not resolve to a

callable target during execution. Consequently, the static analysis query automatically excludes them, ensuring that unmapped optional features do not pollute the call graph with ghost edges.

4.6 Dynamic Validation via bpftrace

While static analysis successfully establishes the theoretically possible execution graph by evaluating source code, it lacks runtime execution context. To verify which of the statically discovered pathways are actively exercised, and to analyze their relative operational frequencies, dynamic tracing was conducted using bpftrace (utilizing kernel kprobes).

The the automatically generated bpftrace (see 3.5.3) was instrumented during an active, real-world decoding workload. To ensure reproducibility and standard compliance, the test application was fed a standardized 10-second, 1080p AV1 encoded video stream (*Sintel*, 10-second snippet, containerized as Sintel_1080_10s_10MB.mp4). The complete quantitative profile detailing the exact execution counts for all tracked implementation functions during this decoding run is organized in Appendix ??.

4.6.1 Subsystem Mathematical Synergy and Workload Analysis

The dynamic profile exhibits tight mathematical coherence, mapping the physical characteristics of the *Sintel* video asset directly onto the structural scheduling patterns of the Linux Memory-to-Memory (m2m) architecture:

- **Deterministic Frame-to-Execution Alignment:** The *Sintel* test video contains exactly 307 encoded AV1 frames (corresponding to a standard encoding profile of roughly 30.7 frames per second over its 10-second duration). This physical asset property perfectly mirrors the driver metrics. The hardware codec driver runner (rockchip_vpu981_av1_dec_run), its completion handler (rockchip_vpu981_av1_dec_done), the core media-to-media job scheduler (device_run), and the frame-level media request handlers (v4l2_ctrl_request_unbind and vb2_req_unbind) all fired exactly 307 times. This absolute $1:1$ correlation provides empirical proof that the trace cleanly isolates frame-boundary executions without dropping or duplicating hardware scheduling jobs.
- **Dual-Queue Symmetric Processing ($2\times$ Multiplier):** Because V4L2 memory-to-memory drivers manage two independent execution queues—an *OUTPUT* queue for incoming compressed AV1 bitstream packets and a *CAPTURE* queue for outgoing raw uncompressed 1080p NV12/YUV video planes—the operational lifecycle boundaries (hantro_queue_setup, hantro_start_streaming,

and `hantro_stop_streaming`) fired exactly 2 times (once per queue). Correspondingly, buffer-level management routines such as `hantro_buf_prepare`, `hantro_buf_queue`, `__fill_vb2_buffer`, and `v4l2_m2m_ioctl_qbuf/dqbuf` all registered exactly 614 executions (307 frames × 2 queues). This perfect proportionality demonstrates that for every frame processed, an operational step is symmetrically mirrored across both streaming topologies.

- **1080p Buffer Pool Allocation and Recycling:** The underlying contiguous memory allocator (`vb2_dc_alloc`) and its accompanying DMA buffer mapping interface (`vb2_dc_get_dmabuf`) both registered exactly 14 invocations. Rather than performing costly dynamic memory reallocations mid-stream for the high-definition 1080p frames, this profile confirms that the application established a static pool of 14 recyclable video buffers at initialization. These 14 hardware-backed memory regions were recursively recycled throughout the 307-frame lifecycle.
- **High-Frequency Data Primitives:** At the lowest level of abstraction, memory tracking and synchronization components fired with compounding frequencies. The buffer synchronization primitive `vb2_dc_cookie` was triggered 2,679 times, illustrating the dense nature of low-level DMA mapping validations occurring continuously beneath the higher-level architectural abstractions to keep the 1080p playback fluent and synchronized.

The absolute mathematical alignment between the physical properties of the video asset and these independent driver structures confirms that the static analysis engine successfully mapped the true execution paths. The data demonstrates that the driver's runtime behavior is a direct, predictable function of the video workload constrained by the V4L2 subsystem's design.

4.7 Cross-Boundary Struct Access Results

4.7.1 Syntactic Analysis

The syntactic field access query returns 34 distinct kernel struct types accessed by driver code across the ioctl-reachable call graph. These span buffer management, format description, control framework, and codec-specific payload structs, and represent the conservative upper bound on the emulation surface.

4.7.2 Taint Tracking

Running the three taint configurations against the same database yields 25 structs with verified ioctl-driven data paths.

D→K returned no results. The kernel receives driver struct pointers — for example via `vb2_set_drv_priv` — but does not reach back into driver-owned memory within the statically traceable call graph. This is a positive finding: the emulation layer does not need to intercept kernel writes to driver-owned state.

K→D confirmed 23 structs accessible through driver callback parameters. These are the buffer lifecycle and streaming control structs passed directly as arguments to registered ops callbacks. Table 4.5 lists the principal types.

Table 4.5: Principal structs accessed via K→D callbacks

Struct	Access	Entry callbacks
`vb2_buffer`	Read	buf_prepare, buf_queue, stop_streaming
`vb2_queue`	Read / Write	buf_prepare, buf_queue, queue_setup
`vb2_v4l2_buffer`	Read / Write	buf_queue, buf_out_validate
`v4l2_pix_format_mplane`	Read / Write	queue_setup, s_fmt callbacks
`v4l2_fh`	Read	buf_queue, start/stop_streaming
`v4l2_m2m_ctx`	Read	start/stop_streaming, s_ctrl
`v4l2_m2m_queue_ctx`	Read	start/stop_streaming
`v4l2_ctrl`	Read	try_ctrl, av1/hevc/vp9 s_ctrl
`media_request_object`	Read	buf_request_complete, stop_streaming

DQ→K confirmed 12 additional structs obtained through driver-initiated kernel accessor calls. These are exclusively codec-specific control structs that the driver retrieves via `hantro_get_ctrl` or the buffer accessors `hantro_get_src_buf` / `hantro_get_dst_buf` before hardware dispatch. Table 4.6 lists the confirmed types grouped by codec.

Table 4.6: Codec-specific structs accessed via DQ→K accessor calls

Struct	Codec	Accessor
`v4l2_ctrl_vp8_frame`	VP8	`hantro_get_ctrl`
`v4l2_vp8_loop_filter`	VP8	`hantro_get_ctrl`
`v4l2_vp8_segment`	VP8	`hantro_get_ctrl`
`v4l2_vp8_quantization`	VP8	`hantro_get_ctrl`
`v4l2_ctrl_vp9_frame`	VP9	`start_prepare_run`
`v4l2_ctrl_vp9_compressed_hdr`	VP9	`start_prepare_run`
`v4l2_vp9_loop_filter`	VP9	`start_prepare_run`
`v4l2_vp9_quantization`	VP9	`start_prepare_run`
`v4l2_vp9_segmentation`	VP9	`start_prepare_run`
`v4l2_ctrl_mpeg2_picture`	MPEG-2	`hantro_get_ctrl`
`v4l2_ctrl_mpeg2_sequence`	MPEG-2	`hantro_get_ctrl`
`v4l2_m2m_buffer`	all	`hantro_get_dst_buf`

4.7.3 Residual Gap and Combined Surface

Nine structs present in the syntactic result are absent from the taint result. All nine are AV1-specific nested types: `v4l2_ctrl_av1_frame`, `v4l2_ctrl_av1_film_grain`, `v4l2_av1_tile_info`, `v4l2_av1_quantization`, `v4l2_av1_loop_filter`, `v4l2_av1_segmentation`, `v4l2_av1_cdef`, `v4l2_av1_loop_restoration`, and `v4l2_av1_global_motion`. Their access path leads through `rockchip_vpu981_av1_dec_prepare_run`, dispatched via the `hantro_codec_ops.run` function pointer. This is a multi-hop indirect call chain that exceeds the resolution depth of the current `resolvesFunctionPointerCall` implementation (Section 3.3.3). Dynamic verification confirms at runtime that these structs are in fact accessed (Section **??**); they are therefore included in the final emulation surface on the strength of the syntactic result, with the taint gap documented as a static analysis limitation.

The 34 structs across both methods form a three-way emulation classification:

1. **Callback-injected** (K→D, 23 structs): must be present as correctly populated arguments in emulation stubs for buffer lifecycle and streaming callbacks.
2. **Query-retrieved** (DQ→K, 12 structs): must be pre-registered as V4L2 controls with valid payload before each hardware invocation, so that `hantro_get_ctrl` returns a live pointer at decode time.
3. **Statically unconfirmed** (9 AV1 nested structs): included conservatively on the basis of syntactic analysis and confirmed by dynamic tracing; their emulation falls under category 2 as they are accessed via the same Request API control path.

This classification is directly actionable for emulation layer design: the distinction between categories 1 and 2 determines not merely which kernel objects to emulate, but how — stub function signatures versus pre-populated control state registered before any ioctl is issued.

5 Conclusion

Lorem ipsum dolor sit amet consectetuer parturient ac pulvinar magna porttitor. Accumsan vel ac eros laoreet Nulla leo Nulla vel Pellentesque Quisque. Adipiscing penatibus Phasellus egestas leo id neque nec quis est orci. Porta tellus ligula ut ridiculus eros eget ut Vivamus dictum nulla. Dui wisi enim vitae nulla Fusce Curabitur congue consectetuer urna Quisque. Felis Vestibulum Quisque sed Vestibulum et malesuada ac id tristique vitae. Aliquam Suspendisse mattis et libero et tincidunt quis tellus eget consectetuer. Libero Morbi cursus augue eget dapibus tincidunt nunc parturient id arcu. Donec sapien enim Aenean convallis Donec elit tincidunt dolor vitae tellus. Ac consectetuer at tortor malesuada ac dui ligula habitant habitasse congue.

List of Figures

List of Tables

List of Algorithms

List of Listings

Bibliography

[AAFX21] Muhammad Abubakar, Adil Ahmad, Pedro Fonseca, and Dongyan Xu: *SHARD: Fine-grained kernel specialization with context-aware hardening.* In *Proceedings of the 30th USENIX Security Symposium*. USENIX Association, 2021.

[All15] Alliance for Open Media: *Alliance for open media established to deliver next-generation open media formats.* Official Press Release, September 2015. `http://aomedia.org/press%20releases/alliance-to-deliver-next-generation-open-media-formats/`.

[Bit25a] Bitmovin Inc.: *8th annual video developer report 2024/2025.* Research report, Bitmovin, February 2025. `https://bitmovin.com/video-developer-report/`, visited on 2026-03-25, Insights from 167 video developers across 34 countries.

[Bit25b] Bitmovin Inc.: *9th annual video developer report 2025/2026.* Research report, Bitmovin, September 2025. `https://bitmovin.com/video-developer-report/`, visited on 2026-03-25, Presented at IBC 2025.

[BS25] Andrin Bertschi and Shweta Shinde: *Opencca: An open framework to enable arm cca research.* In *2025 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*, pages 429–434, 2025.

[BWC+21] Benjamin Bross, Ye Kui Wang, Shanshe Cao, *et al.*: *Overview of the versatile video coding (VVC) standard and its licensing landscape compared to HEVC.* IEEE Transactions on Circuits and Systems for Video Technology, 31(10):3736–3764, 2021.

[CdM23] Arnaldo Carvalho de Melo: *Dwarves and pahole struct layout optimization utilities*, 2023. `https://git.kernel.org/pub/scm/devel/pahole/pahole.git`.

[CMH+20] Yue Chen, Debargha Mukherjee, Jingning Han, Adrian Grange, Yaowu Xu, Sarah Parker, Cheng Chen, Hui Su, Urvang Joshi, Ching Han Chiang, Yunqing Wang, Paul Wilkins, Jim Bankoski, Luc Trudeau, Nathan Egge, Jean Marc Valin, Thomas Davies, Steinar Midtskogen, Andrey Norkin, and Zoe Liu: *An overview of coding tools in av1: the first video codec from the alliance for open media.* APSIPA Transactions on Signal and Information Processing, 9, January 2020.

[CRKH05] Jonathan Corbet, Alessandro Rubini, and Greg Kroah-Hartman: *Linux device drivers, third edition.* O'Reilly Media, Inc., 3rd edition, 2005, ISBN 9780596005900.

[Duf20] Nicolas Dufresne: *Linux stateless video decoder support*, 2020.

[EBSA+12] Hadi Esmaeilzadeh, Emily Blem, Renee St. Amant, Karthikeyan Sankaralingam, and Doug Burger: *Dark silicon and the end of multicore scaling.* IEEE Micro, 32(3):122–134, 2012.

[Ern03] Michael D. Ernst: *Static and dynamic analysis: Synergy and duality.* In *Proceedings of the 1st International Workshop on Dynamic Analysis (WODA)*, pages 24–27. IEEE, 2003.

[Git24] GitHub, Inc.: *Codeql semantic code analysis engine toolchain*, 2024. `https://codeql.github.com/`.

[Git26a] GitHub: *Codeql documentation: Ql language reference*, 2026. `https://codeql.github.com/docs/ql-language-reference/`, Accessed: 2026-05-23.

[Git26b] GitHub: *Codeql library for c and c++*, 2026. `https://codeql.github.com/docs/codeql-language-guides/codeql-library-for-cpp/`, Accessed: 2026-05-23.

[Gre19a] Brendan Gregg: *BPF Performance Tools: Deep Analysis and Visualization.* Addison-Wesley Professional, 2019, ISBN 978-0136554820.

[Gre19b] Brendan Gregg: *BPF Performance Tools: Linux System and Application Observability.* Addison-Wesley Professional, 2019.

[GSt24] GStreamer Developers: *GStreamer v4l2codecs Plugin Documentation.* freedesktop.org, 2024. `https://gstreamer.freedesktop.org/documentation/v4l2codecs/index.html`.

[HND+22] Yongzhe Huang, Vikram Narayanan, David Detweiler, Kaiming Huang, Gang Tan, Trent Jaeger, and Anton Burtsev: *KSplit: Automating device driver isolation.* In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 613–631. USENIX Association, 2022.

[ITU13] ITU-T Recommendation H.265: *High efficiency video coding.* Standard ITU-T H.265 (04/2013), International Telecommunication Union, April 2013. `https://www.itu.int/rec/T-REC-H.265-201304-S/en`.

[J+] Y. Jiang *et al.*: *sNPU: An iommu-backed sandboxing framework for neural processing units.*

[JSF26] Seth Jenkins, Natalie Silvanovich, and Ivan Fratric: *A 0-click exploit chain for the pixel 9: Decoding dolby*. Google Project Zero Security Research Blog, January 2026. https://projectzero.google/.

[Lin24] Linux Manual Pages Project: *proc_kallsyms(5) - Linux manual page*. The Linux Kernel Organization, 2024. https://man7.org/linux/man-pages/man5/proc_kallsyms.5.html.

[Lin26] Linux Kernel Development: *include/uapi/linux/v4l2-controls.h: AV1 stateless control definitions*, 2026. https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/tree/include/uapi/linux/v4l2-controls.h.

[LLV24] LLVM Project: *Clangd Documentation*, 2024. https://clangd.llvm.org/, Version 18.1.

[MDS+24] Lukas Maar, Florian Draschbacher, Lorenz Schumm, Ernesto Martínez García, and Stefan Mangard: *The doom of device drivers: Your android device (most likely) has N-Day kernel vulnerabilities*. In *Proceedings of the 33rd USENIX Security Symposium*. USENIX Association, 2024.

[MPK+06] Ananth Mavinakayanahalli, Prasanna Panchamukhi, Jim Keniston, Anil Keshavamurthy, and Masami Hiramatsu: *Probing the guts of kprobes*. In *Proceedings of the Linux Symposium*, volume 2, pages 109–124, 2006. https://www.kernel.org/doc/ols/2006/ols2006v2-pages-109-124.pdf.

[MS18] Anders Møller and Michael I. Schwartzbach: *Static program analysis*, October 2018. Department of Computer Science, Aarhus University,http://cs.au.dk/~amoeller/spa/.

[NBJ+19] Vikram Narayanan, Abhiram Balasubramanian, Charlie Jacobsen, Sarah Spall, Scott Bauer, Michael Quigley, Aftab Hussain, Abdullah Younis, Junjie Shen, Moinak Bhattacharyya, and Anton Burtsev: *LXDs: Towards isolation of kernel subsystems*. In *Proceedings of the 2019 USENIX Annual Technical Conference (USENIX ATC 19)*, pages 269–284. USENIX Association, 2019.

[Neo24] NeoVim Developer Community: *Neovim: Hyperextensible vim-based text editor*, 2024. https://neovim.io/.

[NNH99] Flemming Nielson, Hanne Nielson, and Chris Hankin: *Principles of Program Analysis*. January 1999, ISBN 978-3-642-08474-4.

[Rad26a] Radxa Computer Co., Ltd.: *ROCK 5B Kernel Development*, 2026. https://docs.radxa.com/en/rock5/rock5b/low-level-dev/kernel, Accessed: May, 2026.

[Rad26b] Radxa Computer Co., Ltd.: *ROCK 5B Product Documentation*, 2026. https://docs.radxa.com/en/rock5/rock5b, Accessed: May 19, 2026.

[Ric04] I.E. Richardson: *H.264 and MPEG-4 Video Compression: Video Coding for Next-generation Multimedia*. Wiley, 2004, ISBN 9780470869604. https://books.google.ch/books?id=n9YVhx2zgz4C.

[Roc24] Rockchip Open Source Community: *Rockchip FIQ Debugger User Guide*. Rockchip Electronics Co., Ltd., June 2024. https://github.com/as-jackson/Rockchip-Docs, Revision V1.2.0.

[Ros26] Steven Rostedt: *Ftrace - Function Tracer*. The Linux Kernel Documentation, 2026. https://www.kernel.org/doc/html/latest/trace/ftrace.html.

[Sil26] Natalie Silvanovich: *A 0-click exploit chain for the pixel 9 part 1: Decoding dolby*, 2026. https://projectzero.google/2026/01/pixel-0-click-part-1.html, visited on 2026-03-21.

[ST26] Knut Sveidqvist and The Mermaid Team: *Mermaid: Markdown-ish text definition syntax for diagram rendering*, 2026. https://mermaid.ai/open-source/intro/index.html.

[Str24] Strace Development Team: *Strace: The diagnostic, debugging and instructional userspace utility for linux*, 2024. https://strace.io/.

[Tha25] Dave Thaler: *Bpf instruction set architecture (isa)*. Internet-draft, IETF BPF Working Group, 2025. https://datatracker.ietf.org/doc/html/draft-ietf-bpf-isa.

[The24a] The Linux Kernel Developers: *BPF Type Format (BTF)*. The Linux Kernel Organization, 2024. https://www.kernel.org/doc/html/v6.12/bpf/btf.html.

[The24b] The Linux Kernel Developers: *Common Clock Framework*. The Linux Kernel Organization, 2024. https://www.kernel.org/doc/html/v6.12/driver-api/clk.html.

[The24c] The Linux Kernel Developers: *Contiguous Memory Allocator (CMA) Debugfs Interface*. The Linux Kernel Organization, 2024. https://www.kernel.org/doc/html/v6.12/admin-guide/mm/cma_debugfs.html.

[The24d] The Linux Kernel Developers: *Devres - Managed Device Resources*. The Linux Kernel Organization, 2024. https://www.kernel.org/doc/html/v6.12/driver-api/driver-model/devres.html.

[The24e] The Linux Kernel Developers: *Dynamic DMA Mapping Guide*. The Linux Kernel Organization, 2024. https://www.kernel.org/doc/html/v6.12/core-api/dma-api.html.

[The24f] The Linux Kernel Developers: *eBPF Documentation*. The Linux Kernel Organization, 2024. https://www.kernel.org/doc/html/v6.12/bpf/index.html.

[The24g] The Linux Kernel Developers: *ioctl VIDIOC_REQBUFS*. The Linux Kernel Organization, 2024. https://www.kernel.org/doc/html/v6.12/userspace-api/media/v4l/vidioc-reqbufs.html.

[The24h] The Linux Kernel Developers: *Kernel Probes (Kprobes)*. The Linux Kernel Organization, 2024. https://www.kernel.org/doc/html/v6.12/trace/kprobes.html.

[The24i] The Linux Kernel Developers: *Linux Kernel Driver Model*. The Linux Kernel Organization, 2024. https://www.kernel.org/doc/html/v6.12/driver-api/driver-model/.

[The24j] The Linux Kernel Developers: *Media Controller Request API*. The Linux Kernel Organization, 2024. https://www.kernel.org/doc/html/v6.12/userspace-api/media/mediactl/request-api.html.

[The24k] The Linux Kernel Developers: *Memory-to-memory Stateless Video Decoder Interface*. The Linux Kernel Organization, 2024. https://www.kernel.org/doc/html/v6.12/userspace-api/media/v4l/dev-stateless-decoder.html.

[The24l] The Linux Kernel Developers: *Platform Devices and Drivers*. The Linux Kernel Organization, 2024. https://www.kernel.org/doc/html/v6.12/driver-api/driver-model/platform.html.

[The24m] The Linux Kernel Developers: *Rockchip Video Decoder Device Tree Bindings*. The Linux Kernel Organization, 2024. https://www.kernel.org/doc/Documentation/devicetree/bindings/media/rockchip,vdec.yaml.

[The24n] The Linux Kernel Developers: *Runtime Power Management Framework*. The Linux Kernel Organization, 2024. https://www.kernel.org/doc/html/v6.12/power/runtime_pm.html.

[The24o] The Linux Kernel Developers: *V4L2 Memory-to-Memory Functions and Data Structures*. The Linux Kernel Organization, 2024. https://www.kernel.org/doc/html/v6.12/driver-api/media/v4l2-mem2mem.html.

[The24p] The Linux Kernel Developers: *Video4Linux Framework*. The Linux Kernel Organization, 2024. https://www.kernel.org/doc/html/v6.12/driver-api/media/v4l2-core.html.

[The24q] The Linux Kernel Developers: *VideoBuffer2 (vb2) Framework*. The Linux Kernel Organization, 2024. `https://www.kernel.org/doc/html/v6.12/driver-api/media/v4l2-videobuf2.html`.

[W+25] Meng Wu *et al.*: *Statically discover cross-entry use-after-free vulnerabilities in the linux kernel*. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2025.

[WLW+17] Zhen Wang, Xi Li, Chao Wang, Zhinan Cheng, Jiachen Song, and Xuehai Zhou: *Rethinking energy-efficiency of heterogeneous computing for cnn-based mobile applications*. In *2017 IEEE International Symposium on Parallel and Distributed Processing with Applications and 2017 IEEE International Conference on Ubiquitous Computing and Communications (ISPA/IUCC)*, pages 454–458, 2017.

[ZBFB14] Jonas Zaddach, Luca Bruno, Aurélien Francillon, and Davide Balzarotti: *Avatar: A framework to support dynamic security analysis of embedded systems' firmwares*. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2014.

A Implementation Code

A.1 Establishing the Research Environment

A.1.1 Kernel Config File

This was appended to the `/bsp/linux/rk2410/0001-common/kconfig.conf` file.

```
# Debugging and Symbol Support
CONFIG_DEBUG_INFO=y
CONFIG_DEBUG_INFO_DWARF5=y
CONFIG_DEBUG_INFO_BTF=y
CONFIG_DEBUG_INFO_BTF_MODULES=y
CONFIG_PAHOLE_HAS_SPLIT_BTF=y
CONFIG_FRAME_POINTER=y
CONFIG_KALLSYMS=y
CONFIG_KALLSYMS_ALL=y

# BPF and Tracing Infrastructure
CONFIG_BPF=y
CONFIG_BPF_EVENTS=y
CONFIG_KPROBES=y
CONFIG_KPROBE_EVENTS=y
CONFIG_TRACING=y
CONFIG_FTRACE=y
CONFIG_FUNCTION_TRACER=y
CONFIG_FUNCTION_GRAPH_TRACER=y
CONFIG_DYNAMIC_FTRACE=y
CONFIG_FTRACE_MCOUNT_RECORD=y
CONFIG_TRACEPOINTS=y
CONFIG_TRACEFS_FS=y

# Ethernet and Driver Support
CONFIG_STMMAC_ETH=y
CONFIG_DWMAC_ROCKCHIP=y
```

Listing A.1: Kernel configuration flags for debugging and tracing

A.1.2 Clangd Configuration File (.clangd)

```
CompileFlags:
  Add:
    - --target=aarch64-linux-gnu
  Remove:
```

```
    # GCC-only arch flags
    - -mabi=lp64
    - -march=armv8-a
    - -mrecord-mcount
    - -mno-outline-atomics
    # GCC-only optimization/codegen flags
    - -fconserve-stack
    - -fno-allow-store-data-races
    - -fno-var-tracking-assignments
    - -fno-ipa-sra
    - --param=allow-store-data-races=0
    # GCC-only warnings
    - -Wno-unused-but-set-variable
    - -Wno-maybe-uninitialized
    - -Wno-packed-not-aligned
    - -Wno-format-truncation
    - -Wno-format-overflow
    - -Wno-frame-address
    - -Wno-alloc-size-larger-than

Diagnostics:
  Suppress:
    - drv_unknown_argument
```

Listing A.2: Full .clangd configuration for ARM64 kernel development

A.1.3 GStreamer 1.24.11 Build

All GStreamer components were built from source to ensure ABI compatibility and support for the `v4l2slav1dec` element. For each of the four core packages (`gstreamer`, `gst-plugins-base`, `good`, and `bad`), the following build sequence was executed from the respective source directory:

```
meson setup build --prefix=/usr/local --wrap-mode=nofallback -Dauto_features=disabled [FLAGS]
ninja -C build -j$(nproc)
sudo ninja -C build install
sudo ldconfig
```

Listing A.3: Meson build process

The specific `[FLAGS]` used for each package are detailed below:

- **Core:** No additional flags.
- **Base:** `-Dvideoconvertscale=enabled -Daudioconvert=enabled -Dtypefind=enabled -Dplayback=enabled -Dapp=enabled -Dpbtypes=enabled`
- **Good:** `-Disomp4=enabled -Dautodetect=enabled`
- **Bad:** `-Dv4l2codecs=enabled -Dvideoparsers=enabled`

A.1.4 GStreamer AV1 Decoding Pipline

```
gst-launch-1.0 filesrc location=~/test.mp4 ! \
qtdemux ! av1parse ! v4l2slav1dec ! fakesink num-buffers=1
```

Listing A.4: Pipline to execute hardware-accelerated AV1 decode

A.2 QL Queries

```
import cpp
import hantro_lib

from Function entry, ExprCall c, Field f, Function callee
where
  Hantro::isV4l2Ioctl(entry) and
  Hantro::callsUnfiltered*(entry, c.getEnclosingFunction()) and
  Hantro::resolvesFunctionPointerCall(c, f, callee) and
  not Hantro::isTrackedOpsStruct(Hantro::fieldStructName(f))
select
  f.getDeclaringType().getName() as ops_struct,
  f.getName() as field,
  c.getEnclosingFunction().getName() as called_from,
  c.getFile().getBaseName() as file,
  c.getLocation().getStartLine() as line,
  callee.getName() as implementation_func
  order by ops_struct, field
```

Listing A.5: Query to identify untracked function pointer implementations

A.3 Results raw

Table A.6: Full cross-module call-edge summary from the CodeQL recursive-call analysis (`ioctl_recursive_calls.ql`). Each row represents one aggregated (*caller module*, *callee module*) pair after grouping individual source files into logical modules. **Max reach** is the highest number of distinct ioctls any single raw query row contributes to that pair; **Ioctls (union)** lists every unique ioctl observed across all raw rows for that pair, sorted alphabetically.

Caller module	**Callee module**	**Max**	
reach	**Ioctls (union)**		
V4L2 mem2mem	V4L2 mem2mem	8	v4l_dqbuf, v4l_qbuf, v4l_querybuf, v4l_reqbufs, v4l_s_ext_ctrls, v4l_s_fmt, v4l_streamoff, v4l_streamon, v4l_stub_expbuf
V4L2 ioctl dispatch	V4L2 core (dev/common)	7	v4l_dqbuf, v4l_enum_fmt, v4l_g_fmt, v4l_qbuf, v4l_query_ext_ctrl, v4l_querybuf, v4l_querycap, v4l_reqbufs, v4l_s_ext_ctrls, v4l_s_fmt
Media controller	VB2 core	6	v4l_dqbuf, v4l_qbuf, v4l_reqbufs, v4l_s_ext_ctrls, v4l_streamoff, v4l_streamon
Media controller	V4L2 controls	6	v4l_dqbuf, v4l_qbuf, v4l_reqbufs, v4l_s_ext_ctrls, v4l_streamoff, v4l_streamon
V4L2 controls	V4L2 controls	6	v4l_qbuf, v4l_query_ext_ctrl, v4l_reqbufs, v4l_s_ext_ctrls, v4l_streamoff, v4l_streamon
VB2 core	VB2 core	6	v4l_dqbuf, v4l_qbuf, v4l_querybuf, v4l_reqbufs, v4l_s_ext_ctrls, v4l_streamoff, v4l_streamon, v4l_stub_expbuf
Hantro driver (generic)	Hantro driver (generic)	5	v4l_enum_fmt, v4l_g_fmt, v4l_qbuf, v4l_reqbufs, v4l_s_ext_ctrls, v4l_s_fmt, v4l_streamoff, v4l_streamon, v4l_stub_enum_framesizes

Continued on next page

Table A.6 – *continued*

Caller module reach	Callee module Ioctls (union)	Max	
V4L2 event	V4L2 event	5	v4l_qbuf, v4l_reqbufs, v4l_s_ext_ctrls, v4l_streamoff, v4l_streamon
Hantro (RK AV1 decode)	Hantro (RK AV1 decode)	4	v4l_qbuf, v4l_reqbufs, v4l_streamoff, v4l_streamon
Hantro driver (generic)	VB2 core	4	v4l_qbuf, v4l_reqbufs, v4l_s_ext_ctrls, v4l_s_fmt, v4l_streamoff, v4l_streamon
Hantro driver (generic)	V4L2 mem2mem	4	v4l_qbuf, v4l_reqbufs, v4l_s_ext_ctrls, v4l_s_fmt, v4l_streamoff, v4l_streamon
Hantro driver (generic)	V4L2 core (dev/common)	4	v4l_qbuf, v4l_querycap, v4l_s_ext_ctrls, v4l_s_fmt, v4l_streamon
V4L2 core (dev/common)	V4L2 core (dev/common)	4	v4l_qbuf, v4l_querycap, v4l_s_ext_ctrls, v4l_s_fmt, v4l_streamon
V4L2 mem2mem	VB2 core	4	v4l_dqbuf, v4l_qbuf, v4l_querybuf, v4l_reqbufs, v4l_streamoff, v4l_streamon, v4l_stub_expbuf
VB2 core	VB2 DMA-contig allocator	4	v4l_dqbuf, v4l_qbuf, v4l_querybuf, v4l_reqbufs, v4l_streamoff, v4l_streamon, v4l_stub_expbuf
V4L2 controls	Hantro driver (generic)	3	v4l_qbuf, v4l_s_ext_ctrls, v4l_streamon
V4L2 controls	V4L2 event	3	v4l_qbuf, v4l_s_ext_ctrls, v4l_streamon
VB2 core	V4L2 core (dev/common)	3	v4l_dqbuf, v4l_qbuf, v4l_querybuf
Hantro (RK AV1 decode)	VB2 core	2	v4l_qbuf, v4l_streamon
Hantro (RK AV1 decode)	VB2 DMA-contig allocator	2	v4l_qbuf, v4l_streamon
Hantro (RK AV1 decode)	Hantro driver (generic)	2	v4l_qbuf, v4l_streamon
Hantro driver (generic)	Hantro (RK AV1 decode)	2	v4l_qbuf, v4l_reqbufs, v4l_streamoff, v4l_streamon

Continued on next page

Table A.6 – *continued*

Caller module reach	Callee module Ioctls (union)	Max	
Hantro driver (generic)	V4L2 controls	2	v4l_qbuf, v4l_reqbufs, v4l_streamoff, v4l_streamon
Hantro driver (generic)	V4L2 event	2	v4l_qbuf, v4l_reqbufs, v4l_streamoff, v4l_streamon
Hantro driver (generic)	VB2 DMA-contig allocator	2	v4l_qbuf, v4l_streamon
MM frame_vector	MM frame_vector	2	v4l_qbuf, v4l_reqbufs
V4L2 mem2mem	Hantro driver (generic)	2	v4l_qbuf, v4l_streamon
VB2 DMA-contig allocator	VB2 DMA-contig allocator	2	v4l_qbuf, v4l_reqbufs, v4l_stub_expbuf
VB2 DMA-contig allocator	VB2 core	2	v4l_qbuf, v4l_streamon
VB2 DMA-contig allocator	MM frame_vector	2	v4l_qbuf, v4l_reqbufs
VB2 core	Hantro driver (generic)	2	v4l_qbuf, v4l_reqbufs, v4l_streamoff, v4l_streamon
V4L2 controls	V4L2 core (dev/common)	1	v4l_s_ext_ctrls
V4L2 ioctl dispatch	V4L2 mem2mem	1	v4l_dqbuf, v4l_qbuf, v4l_querybuf, v4l_reqbufs, v4l_streamoff, v4l_streamon, v4l_stub_expbuf
V4L2 ioctl dispatch	Hantro driver (generic)	1	v4l_enum_fmt, v4l_g_fmt, v4l_querycap, v4l_s_fmt, v4l_stub_enum_framesizes
V4L2 ioctl dispatch	V4L2 ioctl dispatch	1	v4l_dqbuf, v4l_enum_fmt, v4l_g_fmt, v4l_qbuf, v4l_querybuf, v4l_reqbufs, v4l_s_ext_ctrls, v4l_s_fmt
V4L2 ioctl dispatch	V4L2 controls	1	v4l_query_ext_ctrl, v4l_s_ext_ctrls
V4L2 ioctl dispatch	Media controller	1	v4l_s_fmt
V4L2 mem2mem	V4L2 core (dev/common)	1	v4l_qbuf

Table A.7 details the dynamic execution status and precise invocation counts of the hardware-specific implementation functions mapped during the static analysis of the

Rockchip RK3588 AV1 decoding pipeline. The empirical workload was driven by a 10-second, 1080p AV1 clip (`Sintel_1080_10s_10MB.mp4`) containing exactly 307 encoded frames.

Table A.7: Dynamic Quantitative Execution Profile for a 307-Frame (1080p, 10s) AV1 Stream

Function Name	**Associated Subsystem / Ops Struct**	**Dynamic Count / S**
Actively Fired (Confirmed Execution Path)		
vb2_dc_cookie	vb2_mem_ops	2,679
__fill_v4l2_buffer	vb2_buf_ops	1,242
vb2_dc_num_users	vb2_mem_ops	1,242
v4l2_ctrl_type_op_validate	v4l2_ctrl_type_ops	923
v4l2_ctrl_type_op_equal	v4l2_ctrl_type_ops	922
__init_vb2_v4l2_buffer	vb2_buf_ops	628
__copy_timestamp	vb2_buf_ops	614
__fill_vb2_buffer	vb2_buf_ops	614
__verify_planes_array_core	vb2_buf_ops	614
hantro_buf_out_validate	vb2_ops	614
hantro_buf_prepare	vb2_ops	614
hantro_buf_queue	vb2_ops	614
v4l2_m2m_ioctl_dqbuf	v4l2_ioctl_ops	614
v4l2_m2m_ioctl_qbuf	v4l2_ioctl_ops	614
vb2_dc_finish	vb2_mem_ops	614
vb2_dc_prepare	vb2_mem_ops	614
device_run	v4l2_m2m_ops	307
rockchip_vpu981_av1_dec_done	hantro_codec_ops	307
rockchip_vpu981_av1_dec_run	hantro_codec_ops	307
rockchip_vpu981_postproc_disable	Optional HW Feature	307
v4l2_ctrl_request_unbind	media_request_object_ops	307
vb2_req_unbind	media_request_object_ops	307
vb2_dc_put	vb2_mem_ops	42
v4l2_ctrl_type_op_init	v4l2_ctrl_type_ops	31
v4l2_m2m_ioctl_expbuf	v4l2_ioctl_ops	14
v4l2_m2m_ioctl_querybuf	v4l2_ioctl_ops	14
vb2_dc_alloc	vb2_mem_ops	14
vb2_dc_get_dmabuf	vb2_mem_ops	14
vidioc_enum_fmt_vid_cap	v4l2_ioctl_ops	12
vidioc_enum_framesizes	v4l2_ioctl_ops	12
vidioc_g_fmt_cap_mplane	v4l2_ioctl_ops	6
vidioc_enum_fmt_vid_out	v4l2_ioctl_ops	6
vidioc_s_fmt_out_mplane	v4l2_ioctl_ops	5
v4l2_m2m_ioctl_reqbufs	v4l2_ioctl_ops	4

Table A.7: Dynamic Quantitative Execution Profile for a 307-Frame (1080p, 10s) AV1 Stream

Function Name	Associated Subsystem / Ops Struct	Dynamic Coun
hantro_av1_s_ctrl	v4l2_ctrl_ops	3
hantro_try_ctrl	v4l2_ctrl_ops	3
vidioc_querycap	v4l2_ioctl_ops	3
hantro_queue_setup	vb2_ops	2
hantro_start_streaming	vb2_ops	2
hantro_stop_streaming	vb2_ops	2
v4l2_m2m_ioctl_streamoff	v4l2_ioctl_ops	2
v4l2_m2m_ioctl_streamon	v4l2_ioctl_ops	2
rockchip_vpu981_av1_dec_exit	hantro_codec_ops	1
rockchip_vpu981_av1_dec_init	hantro_codec_ops	1
Mapped in Static Analysis, Not Fired dynamically		
hantro_buf_request_complete	vb2_ops	0 (Not Fired)
rockchip_vpu981_postproc_enable	Optional HW Feature	0 (Not Fired)
vb2_dc_attach_dmabuf	vb2_mem_ops	0 (Not Fired)
vb2_dc_detach_dmabuf	vb2_mem_ops	0 (Not Fired)
vb2_dc_get_userptr	vb2_mem_ops	0 (Not Fired)
vb2_dc_map_dmabuf	vb2_mem_ops	0 (Not Fired)
vb2_dc_put_userptr	vb2_mem_ops	0 (Not Fired)
vb2_dc_unmap_dmabuf	vb2_mem_ops	0 (Not Fired)
vb2_ops_wait_finish	vb2_ops	0 (Not Fired)
vb2_ops_wait_prepare	vb2_ops	0 (Not Fired)
Untraceable via Static/Dynamic Tools		
v4l2_ctrl_type_op_log	v4l2_ctrl_type_ops	Not statically fou
vb2_dc_mmap	vb2_mem_ops	Untraceable
vb2_dc_vaddr	vb2_mem_ops	Untraceable